STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID: LRC-26-0050	TEAM NAME: Nova Thinkers.

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids.** Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable.** An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT
Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN
Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (SMPS)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

Team ID: LRC-26-0050 Team Name: NovaThinkers.

HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code.** The case-1 can be turned on using the available on/off Switch, to start the system.

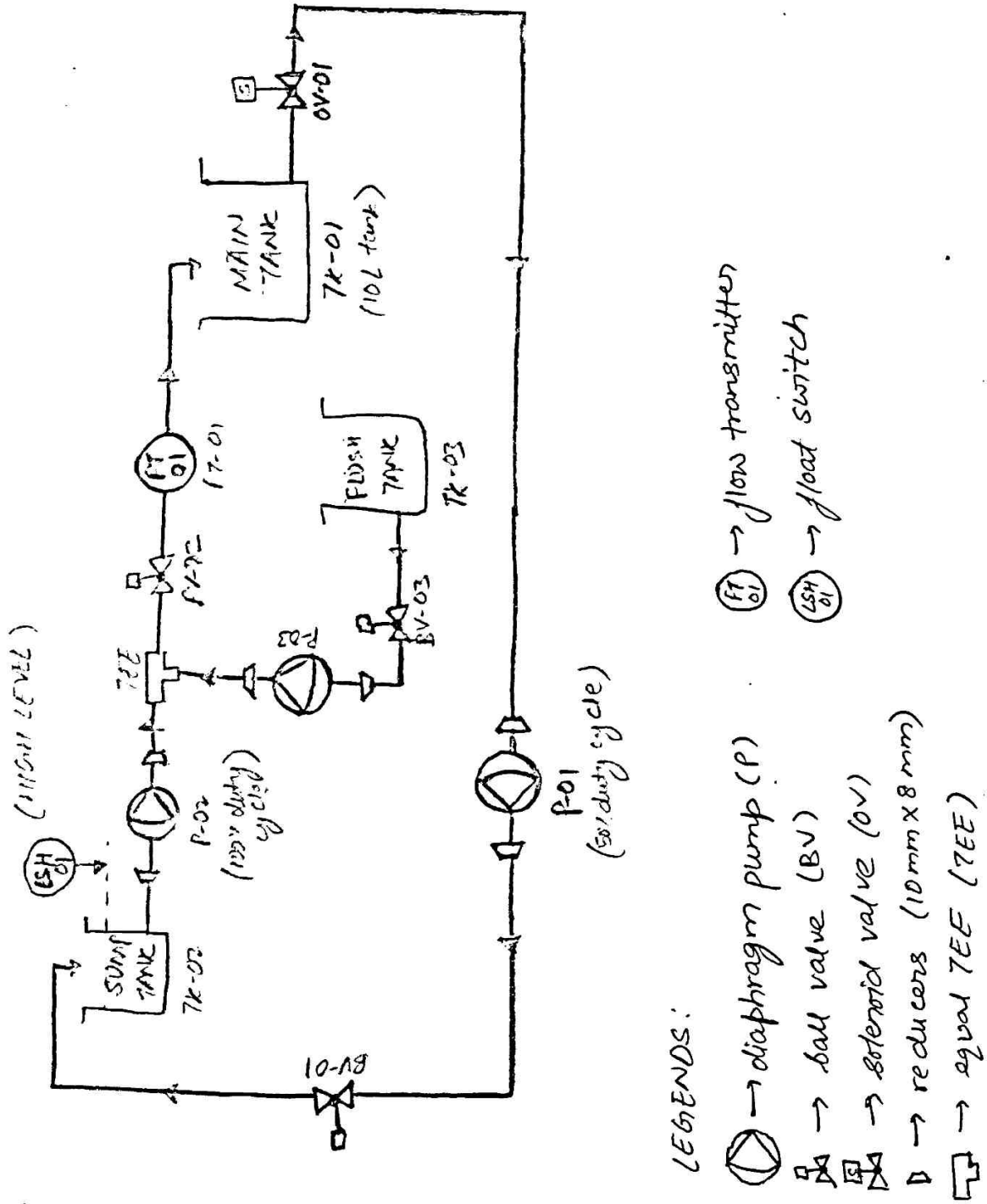
CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram


OV-01
MAIN TANK
TK-01
(10L tank)
FT 01
FV-01
FLUSH TANK
TK-03
EV-03
P-03
ZZZ
LSH 01
SUMP TANK
TK-02
P-02
(Pump used by cycle)
P-01
(50% duty cycle)
10-VB
LEGENDS:
→ diaphragm pump (P)
→ ball valve (BV)
→ solenoid valve (OV)
→ reducers (10mm x 8mm)
→ equal TEE (TEE)
→ flow transmitter
→ float switch

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

Cross-sectional area of given 8 mm diameter pipe;
inner diameter of pipe, $d_i = 5\ mm$

$$A_{cs} = \frac{\pi}{4} d_i^2 = 1.964 \times 10^{-5}\ m^2$$

Hence, flow rate, Q = velocity of flow (v) × cross-sectional area (A_{cs})

Theoretical:

At flow rate, $Q_1 = 1$ lpm;
$v_1 = Q_1 / A_{cs}$
$v_1 = \underline{0.848\ m/s}$.

At $Q_2 = 3$ lpm;
$v_2 = Q_2 / A_{cs}$
$v_2 = \underline{2.546\ m/s}$

At $Q_3 = 5.37$ lpm;
$v_3 = Q_3 / A_{cs}$
$v_3 = \underline{4.554\ m/s}$.

Actual:

Volume of water collected at $Q_1 = 1$ lpm for 1 minute:
$V_1 = 975$ ml. , $(Q_1)_{actual} = \frac{V_1}{1} = \underline{0.975\ lpm}$.

So, actual velocity, $(v_1)_{actual} = \underline{0.827\ m/s}$.

Volume of water collected at $Q_2 = 3$ lpm. for 1 minute:
$V_2 = 3050$ ml , $(Q_2)_{actual} = 3.05$ lpm

∴, actual velocity, $(v_2)_{actual} = \underline{2.586\ m/s}$.

Volume of water collected at $Q_3 = 5.37$ lpm for 1 minute:
$V_3 = 5400$ ml , $(Q_3)_{actual} = 5.4$ lpm.

∴, actual velocity, $(v_3)_{actual} = \underline{4.579\ m/s}$.

$v_1 = \underline{0.848}$ m/s
$(v_1)_{actual} = \underline{0.827}$ m/s.

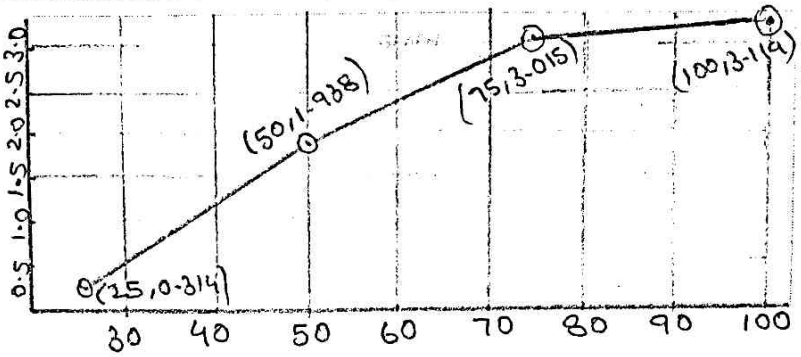
$v_2 = \underline{2.546}$ m/s
$(v_2)_{actual} = \underline{2.586}$ m/s.

$v_3 = \underline{4.554}$ m/s.
$(v_3)_{actual} = \underline{4.579}$ m/s.

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.314
50%	1.938
70%	3.015
100%	3.119



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 = 0.995018

a = 0.02420192

b = 0.02599480

Maximum calibration error = 0.08236 L/min

Average calibration error = 0.05762 L/min

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	752	30	25.067	0.280	0.560
2	1532	30	51.067	0.650	1.300
3	2325	30	77.500	0.992	1.984
4	3149	30	104.967	1.295	2.590
5	3775	30	125.833	1.500	3.000

Trials	f (Hz)	Q (L/min) act	f^2	fQ_{act}	Q (L/min) calc	$\lvert Q_{calc} - Q_{act} \rvert$
1	25.067	0.560	628.354	14.037	0.633	0.07266
2	51.067	1.300	2608.133	66.387	1.262	0.03809
3	77.500	1.984	6006.250	153.760	1.902	0.08236
4	104.967	2.590	11018.067	271.864	2.566	0.02361
5	125.833	3.000	15833.153	377.499	3.071	0.07140

$$a = \frac{n\Sigma f Q_{act} - (\Sigma f)(\Sigma Q_{act})}{n\Sigma f^2 - (\Sigma f)^2}, \quad b = \frac{\Sigma Q_{act} - a\Sigma f}{n}, \quad R^2 = 1 - \frac{\sum_{i=1}^{n}(Q_i - Q_{cal_i})^2}{\sum_{i=1}^{n}(Q_i - \bar{Q})^2}$$

$\Sigma f = 384.4333$, $\Sigma Q_{act} = 9.434$, $\Sigma f^2 = 36094.4211$, $\Sigma f Q_{act} = 883.5477$

$$a = \frac{5(883.5477) - (384.4333)(9.434)}{5(36094.4211) - (384.4333)^2} = 0.02420192$$

$$b = \frac{9.434 - (0.02420192)(384.4333)}{5} = 0.02599489.$$

$$\bar{Q} = \frac{\Sigma Q_i}{n} = \frac{0.560 + 1.300 + 1.984 + 2.590 + 3.000}{5} = 1.88684 \text{ L/min}$$

$$R^2 = 1 - \frac{(0.01916849)}{(3.84788480)} = 1 - 0.004982 = 0.995018.$$

$Q_{1calc} = 0.02420192(25.067) + 0.02599489 = 0.633$
$Q_{2calc} = 0.02420192(51.067) + 0.02599489 = 1.262$
$Q_{3calc} = 0.02420192(77.500) + 0.02599489 = 1.902$
$Q_{4calc} = 0.02420192(104.967) + 0.02599489 = 2.566$
$Q_{5calc} = 0.02420192(125.833) + 0.02599489 = 3.071$

$$MAE = \frac{0.07266 + 0.03809 + 0.08236 + 0.02361 + 0.07140}{5} = 0.05762 \text{ L/min}$$

$$MAE\% = \frac{MAE}{\bar{Q}} \times 100 = \frac{0.05762}{1.8868} \times 100 = 3.05\%$$

HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

Given:- Tubing Volume = 0.5L
Flow rate = 2.5 L/min.

⇒ Residence time = Tubing Volume ÷ Flowrate

$T_R = 0.5 \div 2.5 = 0.2$ minutes = 12 seconds.

1. Physical meaning of Residence time is the average time taken by the liquid to pass through the tubing section. Here, it is 12 sec.

2. If the tubing volume is doubled then the residence time doubles, $T_R' = \frac{2V}{Q} = 2T_R$. (as more liquid must pass through $Q = 24$ sec. at the same flow rate).

3. If the flow rate is doubled then the residence time reduces, $T_R' = \frac{V}{2Q} = \frac{T_R}{2} = 6$ sec. (as the same volume passes through in less time).

4. Importance of residence time when interpreting a sudden change in measured flow is the liquid already inside the tubing takes time to reach the flow transmitter. Hence, a sudden change in flow may be detected after a delay. Understanding residence time helps distinguish normal hydraulic delays from possible pump, blockage or sensor issue.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

A - Pump ON, Valve open, normal pulse and flow: The system is operating normally, with the pump delivering flow and FT-01 measuring the flow correctly.

B - Pump ON, valve partially closed, reduced flow and pulse: The partially closed valve resists the passage of water, increasing flow resistance and reducing the flow rate and pulse frequency.

C - Pump ON, valve open, Zero pulse and flow: This may include pump failure, loss of prime, an empty source tank or a blockage. A faulty sensor or signal connection could also cause a zero pulse even when water is flowing.

D - Pump OFF, Valve open, zero pulse and flow: This is expected condition because the pump is stopped and there is no forced circulation.

E - Pump ON, valve open, intermittent pulses and fluctuating flow: This may be caused by unstable flow, air bubbles, an inconsistent water supply or a loose sensor connection producing a unreliable signal.

- Most difficult condition to diagnose is condition C. Because FT-01 shows zero flow without identifying the cause. I would check the tank level, pump power & operation, pipe pressure & sensor wiring. An independent flow measurement would help confirm when water is actually flowing or FT-01 is failing to detect.

Team ID: LRC-26-0050 Team Name: NovaThinkers.

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces.**
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller.** It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections.**
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

LAM RESEARCH CHALLENGE 3.0 · STAGE 2

Flow Control Experiments

Case 1 and Case 2 operation, and ball-valve opening vs flow rate

Item	Detail
Controller	ESP32 (115200 baud serial link to a Python logger)
Flow instrument	FT-01 pulse flow sensor, Q = 0.02420192·f + 0.0259948 (Q in L/min, f in Hz)
Experiments	Case 1 + Case 2 combined run; ball-valve position vs flow characterization
Experimental runs	11 October 2026
Deliverables	ESP32 firmware, Python logger scripts, CSV data, plots, results

Contents

1. Combined architecture overview (Case 1 and Case 2)
2. Case 1 and Case 2 – ESP32 code
3. Flow-rate and volume calculation
4. Case 1 and Case 2 – Python logging and parsing script
5. Case 1 and Case 2 – final results and graphs
6. Ball valve position vs flow rate – experiment overview
7. Ball valve position vs flow rate – ESP32 code
8. Ball valve position vs flow rate – Python script
9. Ball valve position vs flow rate – results and graphs
10. Interfacing the ESP32 with the Python script over the serial bus

1. Combined Architecture Overview (Case 1 and Case 2)

Case 1 and Case 2 run on one hardware platform and one firmware. An ESP32 microcontroller drives three pumps, a process-tank solenoid valve and reads two inputs – the FT-01 flow sensor and a tank float switch. A physical ON/OFF button and a serial command interface let the operator start, stop and report on the system. Every second the ESP32 converts the FT-01 pulse count to a flow rate, accumulates the transferred volume and streams machine-readable `DATA` and `EVENT` lines to a Python logger on the PC.

1.1 Controller interfaces

Function	ESP32 GPIO	Notes
FT-01 flow sensor	14	Rising-edge interrupt; pulses counted in an ISR
Float switch (tank level)	35	Digital input; triggers Case 1 recovery logic / Case 2 shutdown
Physical ON/OFF button	34	Debounced (50 ms); toggles the system
Process-tank solenoid	27	HIGH = open, LOW = closed
Pump 1 (RPWM/LPWM/REN/LEN)	33 / 32 / 26 / 13	PWM duty 128/255 (≈50 %)
Pump 2 (RPWM/LPWM/REN/LEN)	19 / 18 / 17 / 16	PWM duty 255/255 (100 %)
Pump 3 (RPWM/LPWM/REN/LEN)	25 / 23 / 22 / 21	PWM duty 255/255 (100 %)

Pump PWM runs at 10 kHz with 8-bit resolution.

1.2 How the system works

- **Flow measurement.** FT-01 pulses are counted over a 1 s window to give frequency f (Hz). Flow is $Q = 0.02420192 \cdot f + 0.0259948$ L/min, and the volume added in each window is $Q \cdot \Delta t/60$ litres, accumulated into a running total.
- **Case 1 (normal operation).** The process-tank solenoid is opened, Pump 2 runs at 100 % and Pump 1 at 50 %; Pump 3 stays off. After a 5 s stabilisation buffer the normal-flow average is computed. If the float switch triggers, the solenoid closes while the pumps continue, and an excess/recovery measurement (duration, average flow, volume) runs until the float returns to normal, when the solenoid reopens.
- **Case 2 (flush sequence).** The solenoid is closed and all three pumps run (P1 50 %, P2 100 %, P3 100 %). After a 2 s start-up buffer, the system shuts down automatically if flow falls below 0.35 L/min or if the float switch triggers, then prints the final report.
- **Supervision and safety.** `STOP` or the physical button turns everything off and generates the report immediately. A manual `STOP` locks the system out until the physical button is pressed again.
- **Host side.** A Python logger reads the serial stream, stores raw log, samples and events as CSV, draws the plots and builds a Word report for every run.

1.3 Case 1 vs Case 2 at a glance

	Case 1	Case 2
Solenoid	Open; closes when float triggers	Closed
Pump 1 / 2 / 3	50 % / 100 % / OFF	50 % / 100 % / 100 %
Float switch role	Closes solenoid, starts excess-volume measurement; pumps keep running	Triggers shutdown
Start-up buffer	5 s before normal-flow averaging	2 s before low-flow check

	Case 1	Case 2
Ends when	STOP command or physical button	Flow < 0.35 L/min, float triggered, STOP or button

1.4 Serial commands

`CASE1`, `CASE2`, `STOP`, `STATUS`, `REPORT`, `RESET`, `VALVE`, `HELP` (sent from the Python terminal or any serial terminal at 115200 baud).

2. Case 1 and Case 2 – ESP32 Code

Firmware file: `ESP32_Case1_Case2_Calibrated.ino`. The sketch is organised around a three-state machine (`MODE_OFF`, `MODE_CASE1`, `MODE_CASE2`) that is serviced from `loop()`. The main routines are summarised below; the complete listing follows (blank lines removed to save space).

Routine	Purpose
`flowPulse()` (ISR)	Increments the FT-01 pulse counter on each rising edge
`openSolenoid()` / `closeSolenoid()`	Drive the process-tank solenoid
`startPump1-3()` / `stopPump1-3()` / `stopAllPumps()`	Set PWM duty and enable pins of each pump driver
`startCase1()` / `startCase2()`	Reset measurements and apply the Case 1 / Case 2 pump and solenoid configuration
`updateCase1FloatControl()`	Float trigger/return handling, excess-volume measurement, `EVENT` markers
`updateCase2FloatControl()` / `shutdownCase2()`	Float-triggered shutdown and Case 2 stop sequence with report
`updateFlowMeasurement()`	1 s window: pulses → frequency → flow → volume; prints the `DATA` line; low-flow shutdown check
`startValveMeasurement()` / `updateValveMeasurement()`	Inherited ball-valve routines (4 s stabilisation, 5 s averaging)
`checkStartButton()`	Debounced physical ON/OFF button
`processSerialCommand()`	Serial command interpreter
`printStatus()` / `printFinalReport()`	Status and end-of-run report
`setup()` / `loop()`	Pin, PWM and interrupt initialisation; main scheduler

2.1 Complete listing

```
// ================================================================
// LAM RESEARCH CHALLENGE 3.0
// CASE 1 + CASE 2 DEDICATED SKETCH
//
// Dedicated Case 1/Case 2 use; inherited valve routines remain
available
//
// FEATURES:
// 1. Physical ON/OFF button toggles system
// 2. Case 1 automatic operation
// 3. Case 1 float-switch control
// 4. Automatic excess/recovery volume calculation
// 5. Case 2 flush operation
// 6. FT-01 flow measurement and totalization
// 7. Ball-valve characterization
// 8. STOP command stops + immediately generates report
// 9. Physical button stop also stops + immediately generates report
// 10. REPORT command available at any time
// 11. No continuous FT-01 output while system is OFF
// 12. Ball-valve measurements are reported only when actually measured
// 13. Manual STOP locks system until physical button is pressed again
// ================================================================
// ---------------- PUMP 1 / U2 ----------------
#define P1_RPWM 33
#define P1_LPWM 32
#define P1_REN  26
#define P1_LEN  13
// ---------------- PUMP 2 / U3 ----------------
#define P2_RPWM 19
#define P2_LPWM 18
#define P2_REN  17
#define P2_LEN  16
// ---------------- PUMP 3 / U4 ----------------
#define P3_RPWM 25
#define P3 LPWM 23
#define P3_REN  22
#define P3_LEN  21
// ---------------- OTHER INTERFACES ----------------
#define FLOAT_PIN 35
#define FLOW_PIN  14
#define ONOFF_PIN 34
#define SOLENOID_PIN 27
// ================================================================
// SOLENOID LOGIC
// ================================================================
#define SOLENOID_OPEN_LEVEL   HIGH
#define SOLENOID_CLOSED_LEVEL LOW
// ================================================================
// PWM CONFIGURATION
// ================================================================
#define PWM_FREQ 10000
#define PWM_RES  8
// ================================================================
// PUMP DUTY CYCLES
// ================================================================
#define PUMP1_DUTY 128      // 50%
#define PUMP2_DUTY 255      // 100%
#define PUMP3_DUTY 255      // 100%
const float CAL_A = 0.02420192;
const float CAL_B = 0.02599480;
// ================================================================
// TIMING PARAMETERS
// ================================================================
const unsigned long CASE1_STABLE_BUFFER_MS = 5000;
const unsigned long CASE2_STARTUP_BUFFER_MS = 2000;
const float CASE2_LOW_FLOW_THRESHOLD = 0.35;
const unsigned long SOLENOID_OPEN_DELAY_MS = 1000;
const unsigned long SOLENOID_CLOSE_DELAY_MS = 500;
const unsigned long VALVE_STABILIZATION_MS = 4000;
const unsigned long VALVE_AVERAGING_MS = 5000;
const unsigned long DEBOUNCE_TIME = 50;
// ================================================================
// FLOW VARIABLES
// ================================================================
volatile unsigned long pulseCount = 0;
unsigned long previousFlowMillis = 0;
float currentFlowRate = 0.0;
float totalVolume = 0.0;
// ================================================================
// CASE 1 VARIABLES
// ================================================================
unsigned long case1StartTime = 0;
```

```
float normalFlowSum = 0.0;
unsigned long normalFlowSamples = 0;
float normalFlowAverage = 0.0;
// ================================================================
// EXCESS / RECOVERY VARIABLES
// ================================================================
bool excessFlowEvent = false;
float excessVolume = 0.0;
float totalExcessVolume = 0.0;
float eventStartVolume = 0.0;
unsigned long excessStartTime = 0;
unsigned long excessDuration = 0;
float excessFlowSum = 0.0;
unsigned long excessFlowSamples = 0;
float excessAverageFlow = 0.0;
// ================================================================
// CASE 2 VARIABLES
// ================================================================
unsigned long case2StartTime = 0;
bool case2ShutdownTriggered = false;
// ================================================================
// SYSTEM MODE
// ================================================================
enum SystemMode
{
  MODE_OFF,
  MODE_CASE1,
  MODE_CASE2
};
SystemMode systemMode = MODE_OFF;
// ================================================================
// MANUAL STOP LOCKOUT
// ================================================================
bool manualStopLockout = false;
// ================================================================
// FLOAT SWITCH STATE
// ================================================================
bool lastFloatTriggered = false;
// ================================================================
// PHYSICAL BUTTON VARIABLES
// ================================================================
bool lastButtonState = HIGH;
bool stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;
// ================================================================
// BALL VALVE MEASUREMENT VARIABLES
// ================================================================
bool waitingForValvePosition = false;
bool valveMeasurementActive = false;
int selectedValvePosition = 0;
unsigned long valveStabilizationStart = 0;
unsigned long valveMeasurementStart = 0;
float valveFlowSum = 0.0;
unsigned long valveFlowSamples = 0;
// ================================================================
// BALL VALVE RESULTS
// ================================================================
float flow25 = 0.0;
float flow50 = 0.0;
float flow75 = 0.0;
float flow100 = 0.0;
// Measurement validity flags
bool valve25Measured = false;
bool valve50Measured = false;
bool valve75Measured = false;
bool valve100Measured = false;
// ================================================================
// OPERATING FLOW RANGE
// ================================================================
float minimumOperatingFlow = 9999.0;
float maximumOperatingFlow = 0.0;
// ================================================================
// FLOW SENSOR INTERRUPT
// ================================================================
void IRAM_ATTR flowPulse()
{
  pulseCount++;
}
// ================================================================
// SOLENOID CONTROL
// ================================================================
void openSolenoid()
{
  digitalWrite(
    SOLENOID_PIN,
    SOLENOID_OPEN_LEVEL
  );
  Serial.println(
    "SOLENOID: OPEN SIGNAL SENT"
  );
}
void closeSolenoid()
{
  digitalWrite(
    SOLENOID_PIN,
    SOLENOID_CLOSED_LEVEL
  );
  Serial.println(
    "SOLENOID: CLOSE SIGNAL SENT"
  );
}
// ================================================================
// PUMP 1
// ================================================================
void stopPump1()
{
  ledcWrite(P1_RPWM, 0);
  ledcWrite(P1_LPWM, 0);
  digitalWrite(P1_REN, LOW);
  digitalWrite(P1_LEN, LOW);
}
void startPump1()
{
  digitalWrite(P1_REN, HIGH);
  digitalWrite(P1_LEN, HIGH);
  ledcWrite(P1_LPWM, 0);
  ledcWrite(P1_RPWM, PUMP1_DUTY);
}
// ================================================================
// PUMP 2
// ================================================================
void stopPump2()
{
  ledcWrite(P2_RPWM, 0);
  ledcWrite(P2_LPWM, 0);
  digitalWrite(P2_REN, LOW);
  digitalWrite(P2_LEN, LOW);
}
void startPump2()
{
  digitalWrite(P2_REN, HIGH);
  digitalWrite(P2_LEN, HIGH);
  ledcWrite(P2_RPWM, 0);
  ledcWrite(P2_LPWM, PUMP2_DUTY);
}
// ================================================================
// PUMP 3
// ================================================================
void stopPump3()
{
  ledcWrite(P3_RPWM, 0);
  ledcWrite(P3_LPWM, 0);
  digitalWrite(P3_REN, LOW);
  digitalWrite(P3_LEN, LOW);
}
void startPump3()
{
  digitalWrite(P3_REN, HIGH);
  digitalWrite(P3_LEN, HIGH);
  ledcWrite(P3_LPWM, 0);
  ledcWrite(P3_RPWM, PUMP3_DUTY);
}
// ================================================================
// STOP ALL PUMPS
// ================================================================
void stopAllPumps()
{
  stopPump1();
  stopPump2();
  stopPump3();
}
// ================================================================
// RESET FLOW WINDOW
// ================================================================
void resetFlowWindow()
{
  noInterrupts();
  pulseCount = 0;
  interrupts();
  previousFlowMillis = millis();
}
// ================================================================
// FLOAT SWITCH
// LOW = TRIGGERED
// ================================================================
bool isFloatTriggered()
{
  return digitalRead(FLOAT_PIN) == LOW;
}
// ================================================================
// START CASE 1
// ================================================================
void startCase1()
{
  Serial.println();
  Serial.println(
    "========================================"
  );
  Serial.println(
    "STARTING CASE 1"
  );
  Serial.println(
    "========================================"
  );
  manualStopLockout = false;
  systemMode = MODE_CASE1;
  case1StartTime = millis();
  resetFlowWindow();
  // Reset Case 1 measurements
  normalFlowSum = 0.0;
  normalFlowSamples = 0;
  normalFlowAverage = 0.0;
  // Reset excess measurement
  excessFlowEvent = false;
  excessVolume = 0.0;
  totalExcessVolume = 0.0;
  eventStartVolume = 0.0;
  excessDuration = 0;
  excessFlowSum = 0.0;
  excessFlowSamples = 0;
  excessAverageFlow = 0.0;
  // Reset total volume
  totalVolume = 0.0;
  // Reset operating flow range
```

```
  minimumOperatingFlow = 9999.0;
  maximumOperatingFlow = 0.0;
  lastFloatTriggered =
    isFloatTriggered();
  // ------------------------------------------------
  // OPEN PROCESS TANK SOLENOID
  // ------------------------------------------------
  Serial.println();
  Serial.println(
    "Opening process tank solenoid..."
  );
  openSolenoid();
  delay(
    SOLENOID_OPEN_DELAY_MS
  );
  Serial.println(
    "Solenoid open command completed."
  );
  // ------------------------------------------------
  // START P2
  // ------------------------------------------------
  Serial.println();
  Serial.println(
    "Starting P2 @ 100%..."
  );
  startPump2();
  delay(300);
  // ------------------------------------------------
  // CHECK FLOAT
  // ------------------------------------------------
  if (!isFloatTriggered())
  {
    Serial.println(
      "Float NORMAL."
    );
    Serial.println(
      "Starting P1 @ 50%..."
    );
    startPump1();
  }
  else
  {
    Serial.println(
      "Float TRIGGERED."
    );
    Serial.println(
      "Closing solenoid."
    );
    closeSolenoid();
    Serial.println(
      "Starting P1 @ 50%..."
    );
    startPump1();
  }
  // P3 OFF in Case 1
  stopPump3();
  Serial.println();
  Serial.println(
    "----------------------------------------"
  );
  Serial.println(
    "CASE 1 ACTIVE"
  );
  Serial.println(
    "----------------------------------------"
  );
  Serial.println(
    "SOLENOID = OPEN"
  );
  Serial.println(
    "P1 = 50% when float normal"
  );
  Serial.println(
    "P2 = 100%"
  );
  Serial.println(
    "P3 = OFF"
  );
  Serial.println();
  Serial.println(
    "Float control:"
  );
  Serial.println(
    "NORMAL  -> P1 ON"
  );
  Serial.println(
    "TRIGGER -> SOLENOID CLOSED, P1/P2 CONTINUE"
  );
  Serial.println(
    "RETURN  -> SOLENOID OPENS"
  );
  Serial.println();
  Serial.println(
    "FT-01 measurement ACTIVE"
  );
  Serial.println(
    "----------------------------------------"
  );
}
// ================================================================
// START CASE 2
// ================================================================
void startCase2()
{
  if (manualStopLockout)
  {
    Serial.println();
    Serial.println(
      "========================================"
    );
    Serial.println(
      "CASE 2 BLOCKED"
    );
    Serial.println(
      "========================================"
    );
    Serial.println(
      "System was manually stopped."
    );
    Serial.println(
      "Press physical ON/OFF button"
    );
    Serial.println(
      "to re-arm system."
    );
    Serial.println(
      "========================================"
    );
    return;
  }
  Serial.println();
  Serial.println(
    "STARTING CASE 2 - FLUSH SEQUENCE"
  );
  Serial.println(
    "========================================"
  );
  // Close process tank solenoid
  Serial.println(
    "Closing process tank solenoid..."
  );
  closeSolenoid();
  delay(
    SOLENOID_CLOSE_DELAY_MS
  );
  Serial.println(
    "Solenoid CLOSED."
  );
  systemMode = MODE_CASE2;
  case2StartTime = millis();
  case2ShutdownTriggered = false;
  resetFlowWindow();
  // ------------------------------------------------
  // START CASE 2 PUMPS
  // ------------------------------------------------
  Serial.println(
    "Starting P1 @ 50%..."
  );
  startPump1();
  Serial.println(
    "Starting P2 @ 100%..."
  );
  startPump2();
  Serial.println(
    "Starting P3 @ 100%..."
  );
  startPump3();
  Serial.println();
  Serial.println(
    "----------------------------------------"
  );
  Serial.println(
    "CASE 2 FLUSH ACTIVE"
  );
  Serial.println(
    "----------------------------------------"
  );
  Serial.println(
    "SOLENOID = CLOSED"
  );
  Serial.println(
    "P1 = ON @ 50%"
  );
  Serial.println(
    "P2 = ON @ 100%"
  );
  Serial.println(
    "P3 = ON @ 100%"
  );
  Serial.println();
  Serial.println(
    "FT-01 startup buffer = 2 seconds"
  );
  Serial.println(
    "After buffer:"
  );
  Serial.println(
    "FLOW < 0.35 L/min -> SHUTDOWN"
  );
  Serial.println(
    "----------------------------------------"
  );
}
// ================================================================
// CASE 2 SHUTDOWN
// ================================================================
void shutdownCase2(
  const char *reason
)
{
  if (case2ShutdownTriggered)
    return;
  case2ShutdownTriggered = true;
  Serial.println();
```

```
  Serial.println(
    "========================================"
  );
  Serial.println(
    "CASE 2 SHUTDOWN"
  );
  Serial.println(
    "========================================"
  );
  Serial.print(
    "Reason: "
  );
  Serial.println(
    reason
  );
  Serial.print(
    "Final FT-01 flow: "
  );
  Serial.print(
    currentFlowRate,
    3
  );
  Serial.println(
    " L/min"
  );
  stopAllPumps();
  closeSolenoid();
  systemMode = MODE_OFF;
  Serial.println();
  Serial.println(
    "P1 = OFF"
  );
  Serial.println(
    "P2 = OFF"
  );
  Serial.println(
    "P3 = OFF"
  );
  Serial.println(
    "SOLENOID = CLOSED"
  );
  Serial.println(
    "SYSTEM = OFF"
  );
  Serial.println();
  Serial.println(
    "Generating final report..."
  );
  printFinalReport();
  Serial.println(
    "========================================"
  );
}
// ================================================================
// MANUAL STOP
// ================================================================
//
// This function is called by:
// 1. Physical ON/OFF button while running
// 2. STOP serial command
//
// It immediately:
// - finalizes excess measurement
// - stops all pumps
// - closes solenoid
// - locks system
// - prints report
// ================================================================
void stopSystem()
{
  Serial.println();
  Serial.println(
    "========================================"
  );
  Serial.println(
    "MANUAL SYSTEM STOP"
  );
  Serial.println(
    "========================================"
  );
  // ------------------------------------------------
  // FINALIZE ACTIVE EXCESS EVENT
  // ------------------------------------------------
  if (excessFlowEvent)
  {
    Serial.print("EVENT,FLOAT_NORMAL,");
    Serial.println(millis());
    excessFlowEvent = false;
    excessDuration =
      millis() -
      excessStartTime;
    excessVolume =
      totalVolume -
      eventStartVolume;
    if (excessVolume < 0)
      excessVolume = 0;
    totalExcessVolume +=
      excessVolume;
    if (excessFlowSamples > 0)
    {
      excessAverageFlow =
        excessFlowSum /
        excessFlowSamples;
    }
    else
    {
      excessAverageFlow = 0.0;
    }
    Serial.println();
    Serial.println(
      "----------------------------------------"
    );
    Serial.println(
      "EXCESS / RECOVERY EVENT FINALIZED"
    );
    Serial.println(
      "----------------------------------------"
    );
    Serial.print(
      "Excess duration = "
    );
    Serial.print(
      excessDuration / 1000.0,
      2
    );
    Serial.println(
      " s"
    );
    Serial.print(
      "Excess average flow = "
    );
    Serial.print(
      excessAverageFlow,
      3
    );
    Serial.println(
      " L/min"
    );
    Serial.print(
      "Excess volume = "
    );
    Serial.print(
      excessVolume,
      3
    );
    Serial.println(
      " L"
    );
    Serial.print(
      "Total excess volume = "
    );
    Serial.print(
      totalExcessVolume,
      3
    );
    Serial.println(
      " L"
    );
    Serial.println(
      "----------------------------------------"
    );
  }
  // ------------------------------------------------
  // STOP PHYSICAL OUTPUTS
  // ------------------------------------------------
  stopAllPumps();
  closeSolenoid();
  // ------------------------------------------------
  // STOP BALL VALVE MEASUREMENT
  // ------------------------------------------------
  waitingForValvePosition = false;
  valveMeasurementActive = false;
  selectedValvePosition = 0;
  valveMeasurementStart = 0;
  // ------------------------------------------------
  // SYSTEM OFF + LOCKED
  // ------------------------------------------------
  systemMode = MODE_OFF;
  manualStopLockout = true;
  Serial.println();
  Serial.println(
    "P1 = OFF"
  );
  Serial.println(
    "P2 = OFF"
  );
  Serial.println(
    "P3 = OFF"
  );
  Serial.println(
    "SOLENOID = CLOSED"
  );
  Serial.println(
    "SYSTEM = OFF"
  );
  Serial.println(
    "SYSTEM LOCKED."
  );
  // ------------------------------------------------
  // IMMEDIATE REPORT
  // ------------------------------------------------
  Serial.println();
  Serial.println(
    "Generating current system report..."
  );
  Serial.println();
  printFinalReport();
  Serial.println();
  Serial.println(
    "========================================"
  );
  Serial.println(
    "SYSTEM STOP COMPLETE"
  );
```

```
  Serial.println(
    "========================================"
  );
  Serial.println();
  Serial.println(
    "CASE1 and CASE2 commands are BLOCKED."
  );
  Serial.println(
    "Press physical ON/OFF switch"
  );
  Serial.println(
    "to re-arm and start Case 1."
  );
  Serial.println(
    "========================================"
  );
}
// ================================================================
// CASE 1 FLOAT CONTROL
// ================================================================
void updateCase1FloatControl()
{
  if (systemMode != MODE_CASE1)
    return;
  bool floatTriggered =
    isFloatTriggered();
  // ------------------------------------------------
  // FLOAT JUST TRIGGERED
  // ------------------------------------------------
  if (
    floatTriggered &&
    !lastFloatTriggered
  )
  {
    Serial.println();
    Serial.println(
      "----------------------------------------"
    );
    Serial.println(
      "FLOAT SWITCH TRIGGERED"
    );
    Serial.println(
      "----------------------------------------"
    );
    Serial.println(
      "SOLENOID -> CLOSED"
    );
    Serial.println(
      "P1 -> CONTINUES @ 50%"
    );
    Serial.println(
      "P2 -> CONTINUES @ 100%"
    );
    // Machine-readable event boundary for Python volume integration
    Serial.print("EVENT,FLOAT_TRIGGER,");
    Serial.println(millis());
    // Close solenoid
    closeSolenoid();
    // Start excess measurement
    excessFlowEvent = true;
    eventStartVolume =
      totalVolume;
    excessStartTime =
      millis();
    excessFlowSum = 0.0;
    excessFlowSamples = 0;
    excessVolume = 0.0;
    Serial.println();
    Serial.println(
      "RECOVERY / EXCESS FLOW MEASUREMENT STARTED"
    );
    Serial.println(
      "----------------------------------------"
    );
  }
  // ------------------------------------------------
  // FLOAT REMAINS TRIGGERED
  // ------------------------------------------------
  if (floatTriggered)
  {
    closeSolenoid();
    startPump1();
    startPump2();
  }
  // ------------------------------------------------
  // FLOAT RETURNS TO NORMAL
  // ------------------------------------------------
  if (
    !floatTriggered &&
    lastFloatTriggered
  )
  {
    Serial.println();
    Serial.println(
      "----------------------------------------"
    );
    Serial.println(
      "FLOAT SWITCH RETURNED TO NORMAL"
    );
    Serial.println(
      "----------------------------------------"
    );
    if (excessFlowEvent)
    {
      Serial.print("EVENT,FLOAT_NORMAL,");
      Serial.println(millis());
      excessFlowEvent = false;
      excessDuration =
        millis() -
        excessStartTime;
      excessVolume =
        totalVolume -
        eventStartVolume;
      if (excessVolume < 0)
        excessVolume = 0;
      totalExcessVolume +=
        excessVolume;
      if (excessFlowSamples > 0)
      {
        excessAverageFlow =
          excessFlowSum /
          excessFlowSamples;
      }
      else
      {
        excessAverageFlow = 0.0;
      }
      Serial.print(
        "Excess duration = "
      );
      Serial.print(
        excessDuration / 1000.0,
        2
      );
      Serial.println(
        " s"
      );
      Serial.print(
        "Excess average flow = "
      );
      Serial.print(
        excessAverageFlow,
        3
      );
      Serial.println(
        " L/min"
      );
      Serial.print(
        "Excess volume = "
      );
      Serial.print(
        excessVolume,
        3
      );
      Serial.println(
        " L"
      );
      Serial.print(
        "Total excess volume = "
      );
      Serial.print(
        totalExcessVolume,
        3
      );
      Serial.println(
        " L"
      );
      Serial.println(
        "----------------------------------------"
      );
    }
    // Re-open solenoid
    openSolenoid();
    Serial.println(
      "SOLENOID -> OPEN"
    );
    startPump1();
    startPump2();
    Serial.println(
      "P1 -> CONTINUES @ 50%"
    );
    Serial.println(
      "P2 -> CONTINUES @ 100%"
    );
    Serial.println(
      "----------------------------------------"
    );
  }
  lastFloatTriggered =
    floatTriggered;
}
// ================================================================
// CASE 2 FLOAT CONTROL
// ================================================================
void updateCase2FloatControl()
{
  if (systemMode != MODE_CASE2)
    return;
  if (isFloatTriggered())
  {
    shutdownCase2(
      "FLOAT SWITCH TRIGGERED DURING CASE 2"
    );
  }
}
// ================================================================
// FT-01 FLOW MEASUREMENT
// ================================================================
//
// IMPORTANT:
// When systemMode == MODE_OFF,
// the function does NOT print continuous
// FT-01 information.
//
```

```
// REPORT still gives stored measurements.
// ================================================================
void updateFlowMeasurement()
{
  unsigned long currentMillis =
    millis();
  if (
    currentMillis -
    previousFlowMillis <
    1000
  )
  {
    return;
  }
  unsigned long elapsedMillis =
    currentMillis -
    previousFlowMillis;
  previousFlowMillis =
    currentMillis;
  noInterrupts();
  unsigned long pulses =
    pulseCount;
  pulseCount = 0;
  interrupts();
  float elapsedSeconds =
    elapsedMillis / 1000.0;
  float frequency =
    pulses / elapsedSeconds;
  if (pulses > 0)
  {
    currentFlowRate =
      CAL_A * frequency +
      CAL_B;
    if (currentFlowRate < 0)
      currentFlowRate = 0;
  }
  else
  {
    currentFlowRate = 0;
  }
  // ------------------------------------------------
  // SYSTEM OFF
  // ------------------------------------------------
  if (systemMode == MODE_OFF)
  {
    return;
  }
  // ------------------------------------------------
  // VOLUME
  // ------------------------------------------------
  float volumeThisInterval =
    currentFlowRate *
    elapsedSeconds /
    60.0;
  totalVolume +=
    volumeThisInterval;
  // Machine-readable row consumed by lam_stage2_logger.py.
  // Fields:
DATA,millis,mode,pulses,frequency_hz,flow_l_min,total_volume_l,float,eve
nt,solenoid
  Serial.print("DATA,");
  Serial.print(currentMillis);
  Serial.print(",");
  if (systemMode == MODE_CASE1) Serial.print("CASE1");
  else if (systemMode == MODE_CASE2) Serial.print("CASE2");
  else Serial.print("OFF");
  Serial.print(",");
  Serial.print(pulses);
  Serial.print(",");
  Serial.print(frequency, 4);
  Serial.print(",");
  Serial.print(currentFlowRate, 5);
  Serial.print(",");
  Serial.print(totalVolume, 6);
  Serial.print(",");
  Serial.print(isFloatTriggered() ? 1 : 0);
  Serial.print(",");
  Serial.print(excessFlowEvent ? 1 : 0);
  Serial.print(",");
  Serial.println(digitalRead(SOLENOID_PIN) == SOLENOID_OPEN_LEVEL ? 1 :
0);
  // ------------------------------------------------
  // OPERATING FLOW RANGE
  // ------------------------------------------------
  if (currentFlowRate > 0)
  {
    if (
      currentFlowRate <
      minimumOperatingFlow
    )
    {
      minimumOperatingFlow =
        currentFlowRate;
    }
    if (
      currentFlowRate >
      maximumOperatingFlow
    )
    {
      maximumOperatingFlow =
        currentFlowRate;
    }
  }
  // ------------------------------------------------
  // CASE 1 MEASUREMENT
  // ------------------------------------------------
  if (systemMode == MODE_CASE1)
  {
    unsigned long case1Elapsed =
      currentMillis -
      case1StartTime;
    // Normal flow averaging
    if (
      !isFloatTriggered() &&
      !excessFlowEvent &&
      case1Elapsed >=
      CASE1_STABLE_BUFFER_MS
    )
    {
      normalFlowSum +=
        currentFlowRate;
      normalFlowSamples++;
      if (normalFlowSamples > 0)
      {
        normalFlowAverage =
          normalFlowSum /
          normalFlowSamples;
      }
    }
    // Excess flow averaging
    if (excessFlowEvent)
    {
      excessFlowSum +=
        currentFlowRate;
      excessFlowSamples++;
      excessVolume =
        totalVolume -
        eventStartVolume;
    }
  }
  // ------------------------------------------------
  // LIVE FT-01 DISPLAY
  // ------------------------------------------------
  Serial.println();
  Serial.println(
    "--------------- FT-01 ----------------"
  );
  Serial.print(
    "Pulses       : "
  );
  Serial.println(
    pulses
  );
  Serial.print(
    "Frequency    : "
  );
  Serial.print(
    frequency,
    3
  );
  Serial.println(
    " Hz"
  );
  Serial.print(
    "Flow rate    : "
  );
  Serial.print(
    currentFlowRate,
    3
  );
  Serial.println(
    " L/min"
  );
  Serial.print(
    "Total volume : "
  );
  Serial.print(
    totalVolume,
    3
  );
  Serial.println(
    " L"
  );
  if (systemMode == MODE_CASE1)
  {
    Serial.print(
      "Normal avg   : "
    );
    Serial.print(
      normalFlowAverage,
      3
    );
    Serial.println(
      " L/min"
    );
    if (excessFlowEvent)
    {
      Serial.print(
        "Excess volume: "
      );
      Serial.print(
        excessVolume,
        3
      );
      Serial.println(
        " L"
      );
    }
  }
  Serial.println(
    "----------------------------------------"
  );
  // ------------------------------------------------
  // CASE 2 FLOW LOGIC
```

```
  // ------------------------------------------------
  if (systemMode == MODE_CASE2)
  {
    unsigned long case2Elapsed =
      currentMillis -
      case2StartTime;
    // Startup buffer
    if (
      case2Elapsed <
      CASE2_STARTUP_BUFFER_MS
    )
    {
      unsigned long remaining =
        CASE2_STARTUP_BUFFER_MS -
        case2Elapsed;
      Serial.print(
        "CASE 2 startup buffer: "
      );
      Serial.print(
        remaining / 1000.0,
        1
      );
      Serial.println(
        " s remaining"
      );
      return;
    }
    // Low-flow shutdown
    if (
      currentFlowRate <
      CASE2_LOW_FLOW_THRESHOLD
    )
    {
      Serial.println();
      Serial.println(
        "!!! CASE 2 LOW FLOW !!!"
      );
      Serial.print(
        "Measured flow = "
      );
      Serial.print(
        currentFlowRate,
        3
      );
      Serial.println(
        " L/min"
      );
      Serial.print(
        "Shutdown threshold = "
      );
      Serial.print(
        CASE2_LOW_FLOW_THRESHOLD,
        2
      );
      Serial.println(
        " L/min"
      );
      shutdownCase2(
        "FT-01 FLOW BELOW 0.35 L/min"
      );
      return;
    }
    Serial.println(
      "CASE 2: FLOW ABOVE 0.35 L/min"
    );
  }
}
// ================================================================
// START BALL VALVE MEASUREMENT
// ================================================================
void startValveMeasurement(
  int position
)
{
  selectedValvePosition =
    position;
  waitingForValvePosition =
    false;
  valveMeasurementActive =
    true;
  valveStabilizationStart =
    millis();
  valveMeasurementStart =
    0;
  valveFlowSum =
    0.0;
  valveFlowSamples =
    0;
  Serial.println();
  Serial.println(
    "========================================"
  );
  Serial.print(
    "BALL VALVE POSITION = "
  );
  Serial.print(
    position
  );
  Serial.println(
    "%"
  );
  Serial.println(
    "========================================"
  );
  Serial.println(
    "Stabilizing for 4 seconds..."
  );
}
// ================================================================
// UPDATE BALL VALVE MEASUREMENT
// ================================================================
void updateValveMeasurement()
{
  if (!valveMeasurementActive)
    return;
  unsigned long now =
    millis();
  // ------------------------------------------------
  // STABILIZATION
  // ------------------------------------------------
  if (
    now -
    valveStabilizationStart <
    VALVE_STABILIZATION_MS
  )
  {
    return;
  }
  // ------------------------------------------------
  // START AVERAGING
  // ------------------------------------------------
  if (valveMeasurementStart == 0)
  {
    valveMeasurementStart =
      now;
    valveFlowSum =
      0.0;
    valveFlowSamples =
      0;
    Serial.println(
      "Stabilization complete."
    );
    Serial.println(
      "Averaging flow for 5 seconds..."
    );
    return;
  }
  // ------------------------------------------------
  // AVERAGING
  // ------------------------------------------------
  if (
    now -
    valveMeasurementStart <
    VALVE_AVERAGING_MS
  )
  {
    valveFlowSum +=
      currentFlowRate;
    valveFlowSamples++;
    return;
  }
  // ------------------------------------------------
  // CALCULATE RESULT
  // ------------------------------------------------
  float averageFlow =
    0.0;
  if (valveFlowSamples > 0)
  {
    averageFlow =
      valveFlowSum /
      valveFlowSamples;
  }
  // ------------------------------------------------
  // SAVE RESULT
  // ------------------------------------------------
  if (selectedValvePosition == 25)
  {
    flow25 =
      averageFlow;
    valve25Measured =
      true;
  }
  else if (selectedValvePosition == 50)
  {
    flow50 =
      averageFlow;
    valve50Measured =
      true;
  }
  else if (selectedValvePosition == 75)
  {
    flow75 =
      averageFlow;
    valve75Measured =
      true;
  }
  else if (selectedValvePosition == 100)
  {
    flow100 =
      averageFlow;
    valve100Measured =
      true;
  }
  Serial.println();
  Serial.println(
    "----------------------------------------"
  );
  Serial.print(
    "BALL VALVE "
  );
  Serial.print(
    selectedValvePosition
  );
  Serial.println(
```

```
    "% RESULT"
  );
  Serial.print(
    "Stable flow = "
  );
  Serial.print(
    averageFlow,
    3
  );
  Serial.println(
    " L/min"
  );
  Serial.println(
    "----------------------------------------"
  );
  valveMeasurementActive =
    false;
  valveMeasurementStart =
    0;
  selectedValvePosition =
    0;
}
// ================================================================
// PHYSICAL ON/OFF BUTTON
// ================================================================
//
// OFF + button:
//     Start Case 1
//
// RUNNING + button:
//     Stop system + report
// ================================================================
void checkStartButton()
{
  bool reading =
    digitalRead(
      ONOFF_PIN
    );
  // Debounce
  if (
    reading !=
    lastButtonState
  )
  {
    lastDebounceTime =
      millis();
  }
  if (
    millis() -
    lastDebounceTime >
    DEBOUNCE_TIME
  )
  {
    if (
      reading !=
      stableButtonState
    )
    {
      stableButtonState =
        reading;
      // Button pressed
      if (
        stableButtonState ==
        LOW
      )
      {
        Serial.println();
        Serial.println(
          "PHYSICAL ON/OFF BUTTON PRESSED"
        );
        // ----------------------------------------
        // SYSTEM OFF
        // ----------------------------------------
        if (
          systemMode ==
          MODE_OFF
        )
        {
          manualStopLockout =
            false;
          startCase1();
        }
        // ----------------------------------------
        // SYSTEM RUNNING
        // ----------------------------------------
        else
        {
          stopSystem();
        }
      }
    }
  }
  lastButtonState =
    reading;
}
// ================================================================
// STATUS
// ================================================================
void printStatus()
{
  Serial.println();
  Serial.println(
    "========================================"
  );
  Serial.println(
    "SYSTEM STATUS"
  );
  Serial.println(
    "========================================"
  );
  Serial.print(
    "Mode: "
  );
  if (
    systemMode ==
    MODE_OFF
  )
  {
    Serial.println(
      "OFF"
    );
  }
  else if (
    systemMode ==
    MODE_CASE1
  )
  {
    Serial.println(
      "CASE 1"
    );
  }
  else if (
    systemMode ==
    MODE_CASE2
  )
  {
    Serial.println(
      "CASE 2"
    );
  }
  Serial.print(
    "STOP Lockout: "
  );
  if (manualStopLockout)
    Serial.println("ACTIVE");
  else
    Serial.println("NO");
  Serial.print(
    "Solenoid command: "
  );
  if (
    digitalRead(
      SOLENOID_PIN
    ) ==
    SOLENOID_OPEN_LEVEL
  )
  {
    Serial.println(
      "OPEN"
    );
  }
  else
  {
    Serial.println(
      "CLOSED"
    );
  }
  Serial.print(
    "Float: "
  );
  if (isFloatTriggered())
    Serial.println("TRIGGERED");
  else
    Serial.println("NORMAL");
  Serial.print(
    "Flow: "
  );
  Serial.print(
    currentFlowRate,
    3
  );
  Serial.println(
    " L/min"
  );
  Serial.print(
    "Total volume: "
  );
  Serial.print(
    totalVolume,
    3
  );
  Serial.println(
    " L"
  );
  Serial.println();
  Serial.println(
    "PUMP STATUS:"
  );
  if (
    systemMode ==
    MODE_CASE1
  )
  {
    if (isFloatTriggered())
    {
      Serial.println(
        "P1 = ON @ 50% - FLOAT TRIGGERED"
      );
    }
    else
    {
      Serial.println(
        "P1 = ON @ 50%"
      );
```

```
    }
    Serial.println(
      "P2 = ON @ 100%"
    );
    Serial.println(
      "P3 = OFF"
    );
  }
  else if (
    systemMode ==
    MODE_CASE2
  )
  {
    Serial.println(
      "P1 = ON @ 50%"
    );
    Serial.println(
      "P2 = ON @ 100%"
    );
    Serial.println(
      "P3 = ON @ 100%"
    );
  }
  else
  {
    Serial.println(
      "P1 = OFF"
    );
    Serial.println(
      "P2 = OFF"
    );
    Serial.println(
      "P3 = OFF"
    );
  }
  Serial.println(
    "============================================"
  );
}
// ================================================================
// FINAL / CURRENT SYSTEM REPORT
// ================================================================
//
// This report NEVER waits for ball-valve testing.
//
// Unmeasured valve positions are shown as
// "NOT MEASURED".
//
// Can be called:
// - automatically after STOP
// - automatically after physical button STOP
// - automatically after Case 2 shutdown
// - manually using REPORT
// ================================================================
void printFinalReport()
{
  Serial.println();
  Serial.println();
  Serial.println(
    "================================================"
  );
  Serial.println(
    "              CURRENT SYSTEM REPORT"
  );
  Serial.println(
    "================================================"
  );
  // ============================================================
  // FT-01 CALIBRATION
  // ============================================================
  Serial.println();
  Serial.println(
    "FT-01 CALIBRATION"
  );
  Serial.println(
    "---------------------------------"
  );
  Serial.print(
    "Q = "
  );
  Serial.print(
    CAL_A,
    8
  );
  Serial.print(
    " * f + "
  );
  Serial.println(
    CAL_B,
    7
  );
  // ============================================================
  // CASE 1 NORMAL
  // ============================================================
  Serial.println();
  Serial.println(
    "CASE 1 - NORMAL OPERATION"
  );
  Serial.println(
    "---------------------------------"
  );
  if (
    normalFlowSamples > 0
  )
  {
    Serial.print(
      "Normal sustained flow = "
    );
    Serial.print(
      normalFlowAverage,
      3
    );
    Serial.println(
      " L/min"
    );
  }
  else
  {
    Serial.println(
      "Normal sustained flow = NOT MEASURED"
    );
  }
  // ============================================================
  // CASE 1 EXCESS / RECOVERY
  // ============================================================
  Serial.println();
  Serial.println(
    "CASE 1 - EXCESS / RECOVERY"
  );
  Serial.println(
    "---------------------------------"
  );
  if (
    totalExcessVolume > 0 ||
    excessDuration > 0
  )
  {
    Serial.print(
      "Total excess volume = "
    );
    Serial.print(
      totalExcessVolume,
      3
    );
    Serial.println(
      " L"
    );
    Serial.print(
      "Last excess duration = "
    );
    Serial.print(
      excessDuration / 1000.0,
      2
    );
    Serial.println(
      " s"
    );
    Serial.print(
      "Last excess average flow = "
    );
    Serial.print(
      excessAverageFlow,
      3
    );
    Serial.println(
      " L/min"
    );
  }
  else
  {
    Serial.println(
      "Excess event = NOT MEASURED"
    );
  }
  // ============================================================
  // BALL VALVE CHARACTERIZATION
  // ============================================================
  Serial.println();
  Serial.println(
    "BALL VALVE CHARACTERIZATION"
  );
  Serial.println(
    "---------------------------------"
  );
  // 25%
  Serial.print(
    "25%  = "
  );
  if (valve25Measured)
  {
    Serial.print(
      flow25,
      3
    );
    Serial.println(
      " L/min"
    );
  }
  else
  {
    Serial.println(
      "NOT MEASURED"
    );
  }
  // 50%
  Serial.print(
    "50%  = "
  );
  if (valve50Measured)
  {
    Serial.print(
      flow50,
      3
    );
```

```
  Serial.println(
    " L/min"
  );
}
else
{
  Serial.println(
    "NOT MEASURED"
  );
}
// 75%
Serial.print(
  "75%  = "
);
if (valve75Measured)
{
  Serial.print(
    flow75,
    3
  );
  Serial.println(
    " L/min"
  );
}
else
{
  Serial.println(
    "NOT MEASURED"
  );
}
// 100%
Serial.print(
  "100% = "
);
if (valve100Measured)
{
  Serial.print(
    flow100,
    3
  );
  Serial.println(
    " L/min"
  );
}
else
{
  Serial.println(
    "NOT MEASURED"
  );
}
// ============================================================
// OPERATING FLOW RANGE
// ============================================================
Serial.println();
Serial.println(
  "TOTAL OPERATING FLOW RANGE"
);
Serial.println(
  "--------------------------------"
);
if (
  minimumOperatingFlow <
  9999.0
)
{
  Serial.print(
    "Minimum = "
  );
  Serial.print(
    minimumOperatingFlow,
    3
  );
  Serial.println(
    " L/min"
  );
  Serial.print(
    "Maximum = "
  );
  Serial.print(
    maximumOperatingFlow,
    3
  );
  Serial.println(
    " L/min"
  );
  Serial.print(
    "Range = "
  );
  Serial.print(
    maximumOperatingFlow -
    minimumOperatingFlow,
    3
  );
  Serial.println(
    " L/min"
  );
}
else
{
  Serial.println(
    "No valid operating-flow data."
  );
}
// ============================================================
// TOTAL FT-01 VOLUME
// ============================================================
Serial.println();
Serial.println(
  "TOTAL FT-01 VOLUME"
);
Serial.println(
  "--------------------------------"
);
Serial.print(
  "Total measured volume = "
);
Serial.print(
  totalVolume,
  3
);
Serial.println(
  " L"
);
// ============================================================
// FINAL STATE
// ============================================================
Serial.println();
Serial.println(
  "FINAL STATE"
);
Serial.println(
  "--------------------------------"
);
if (
  systemMode ==
  MODE_OFF
)
{
  Serial.println(
    "P1 = OFF"
  );
  Serial.println(
    "P2 = OFF"
  );
  Serial.println(
    "P3 = OFF"
  );
  Serial.println(
    "SOLENOID = CLOSED"
  );
  Serial.println(
    "SYSTEM = OFF"
  );
}
else if (
  systemMode ==
  MODE_CASE1
)
{
  Serial.println(
    "P1 = ON @ 50%"
  );
  Serial.println(
    "P2 = ON @ 100%"
  );
  Serial.println(
    "P3 = OFF"
  );
  Serial.println(
    "SYSTEM = CASE 1 ACTIVE"
  );
}
else if (
  systemMode ==
  MODE_CASE2
)
{
  Serial.println(
    "P1 = ON @ 50%"
  );
  Serial.println(
    "P2 = ON @ 100%"
  );
  Serial.println(
    "P3 = ON @ 100%"
  );
  Serial.println(
    "SYSTEM = CASE 2 ACTIVE"
  );
}
Serial.println();
Serial.println(
  "================================================"
);
Serial.println(
  "               REPORT COMPLETE"
);
Serial.println(
  "================================================"
);
}
// ============================================================
// RESET ALL MEASUREMENTS
// ============================================================
void resetMeasurements()
{
  noInterrupts();
  pulseCount = 0;
  interrupts();
  totalVolume = 0.0;
  currentFlowRate = 0.0;
  normalFlowSum = 0.0;
  normalFlowSamples = 0;
  normalFlowAverage = 0.0;
```

```
  excessFlowEvent = false;
  excessVolume = 0.0;
  totalExcessVolume = 0.0;
  eventStartVolume = 0.0;
  excessDuration = 0;
  excessFlowSum = 0.0;
  excessFlowSamples = 0;
  excessAverageFlow = 0.0;
  minimumOperatingFlow = 9999.0;
  maximumOperatingFlow = 0.0;
  flow25 = 0.0;
  flow50 = 0.0;
  flow75 = 0.0;
  flow100 = 0.0;
  valve25Measured = false;
  valve50Measured = false;
  valve75Measured = false;
  valve100Measured = false;
  Serial.println();
  Serial.println(
    "ALL FLOW MEASUREMENTS RESET."
  );
}
// ================================================================
// SERIAL COMMAND PROCESSING
// ================================================================
void processSerialCommand()
{
  if (!Serial.available())
    return;
  String command =
    Serial.readStringUntil('\n');
  command.trim();
  command.toUpperCase();
  // ============================================================
  // CASE 1
  // ============================================================
  if (command == "CASE1")
  {
    if (manualStopLockout)
    {
      Serial.println();
      Serial.println(
        "CASE1 BLOCKED."
      );
      Serial.println(
        "System was stopped using STOP."
      );
      Serial.println(
        "Press physical ON/OFF switch first."
      );
    }
    else if (
      systemMode ==
      MODE_OFF
    )
    {
      startCase1();
    }
    else
    {
      Serial.println(
        "System already running."
      );
    }
    return;
  }
  // ============================================================
  // CASE 2
  // ============================================================
  if (command == "CASE2")
  {
    startCase2();
    return;
  }
  // ============================================================
  // STOP
  // ============================================================
  if (command == "STOP")
  {
    if (systemMode != MODE_OFF)
    {
      stopSystem();
    }
    else
    {
      Serial.println();
      Serial.println(
        "SYSTEM ALREADY OFF."
      );
      Serial.println(
        "Generating current report..."
      );
      printFinalReport();
    }
    return;
  }
  // ============================================================
  // STATUS
  // ============================================================
  if (command == "STATUS")
  {
    printStatus();
    return;
  }
  // ============================================================
  // REPORT
  // ============================================================
  if (command == "REPORT")
  {
    printFinalReport();
    return;
  }
  // ============================================================
  // RESET
  // ============================================================
  if (command == "RESET")
  {
    resetMeasurements();
    return;
  }
  // ============================================================
  // VALVE
  // ============================================================
  if (command == "VALVE")
  {
    Serial.println();
    Serial.println(
      "========================================"
    );
    Serial.println(
      "BALL VALVE FLOW MEASUREMENT"
    );
    Serial.println(
      "========================================"
    );
    Serial.println(
      "Enter valve position:"
    );
    Serial.println(
      "25"
    );
    Serial.println(
      "50"
    );
    Serial.println(
      "75"
    );
    Serial.println(
      "100"
    );
    Serial.println();
    waitingForValvePosition =
      true;
    return;
  }
  // ============================================================
  // VALVE POSITION
  // ============================================================
  if (waitingForValvePosition)
  {
    int position =
      command.toInt();
    if (
      position == 25 ||
      position == 50 ||
      position == 75 ||
      position == 100
    )
    {
      waitingForValvePosition =
        false;
      startValveMeasurement(
        position
      );
    }
    else
    {
      Serial.println(
        "Invalid valve position."
      );
      Serial.println(
        "Enter 25, 50, 75 or 100."
      );
    }
    return;
  }
  // ============================================================
  // HELP
  // ============================================================
  if (command == "HELP")
  {
    Serial.println();
    Serial.println(
      "========================================"
    );
    Serial.println(
      "AVAILABLE COMMANDS"
    );
    Serial.println(
      "========================================"
    );
    Serial.println(
      "CASE1  -> Start Case 1"
    );
    Serial.println(
      "CASE2  -> Start Case 2"
    );
    Serial.println(
      "STOP   -> Stop + generate report"
    );
    Serial.println(
      "STATUS -> Show system status"
    );
```

```
    Serial.println(
      "REPORT -> Show current report"
    );
    Serial.println(
      "RESET  -> Reset measurements"
    );
    Serial.println(
      "VALVE  -> Measure ball valve flow"
    );
    Serial.println(
      "HELP   -> Show commands"
    );
    Serial.println(
      "========================================="
    );
    return;
  }
  // ============================================================
  // UNKNOWN COMMAND
  // ============================================================
  Serial.print(
    "Unknown command: "
  );
  Serial.println(
    command
  );
  Serial.println(
    "Type HELP for commands."
  );
}
// ============================================================
// SETUP
// ============================================================
void setup()
{
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println();
  Serial.println(
    "================================================"
  );
  Serial.println(
    "       LAM RESEARCH CHALLENGE 3.0"
  );
  Serial.println(
    "       STAGE 2 ESP32 CONTROL SYSTEM"
  );
  Serial.println(
    "================================================"
  );
  // ============================================================
  // PIN MODES
  // ============================================================
  pinMode(P1_RPWM, OUTPUT);
  pinMode(P1_LPWM, OUTPUT);
  pinMode(P1_REN, OUTPUT);
  pinMode(P1_LEN, OUTPUT);
  pinMode(P2_RPWM, OUTPUT);
  pinMode(P2_LPWM, OUTPUT);
  pinMode(P2_REN, OUTPUT);
  pinMode(P2_LEN, OUTPUT);
  pinMode(P3_RPWM, OUTPUT);
  pinMode(P3_LPWM, OUTPUT);
  pinMode(P3_REN, OUTPUT);
  pinMode(P3_LEN, OUTPUT);
  pinMode(
    FLOAT_PIN,
    INPUT
  );
  pinMode(
    FLOW_PIN,
    INPUT
  );
  pinMode(
    ONOFF_PIN,
    INPUT
  );
  pinMode(
    SOLENOID_PIN,
    OUTPUT
  );
  // ============================================================
  // SOLENOID SAFE POWER-UP STATE
  // ============================================================
  digitalWrite(
    SOLENOID_PIN,
    SOLENOID_CLOSED_LEVEL
  );
  // ============================================================
  // PWM
  // ============================================================
  ledcAttach(
    P1_RPWM,
    PWM_FREQ,
    PWM_RES
  );
  ledcAttach(
    P1_LPWM,
    PWM_FREQ,
    PWM_RES
  );
  ledcAttach(
    P2_RPWM,
    PWM_FREQ,
    PWM_RES
  );
  ledcAttach(
    P2_LPWM,
    PWM_FREQ,
    PWM_RES
  );
  ledcAttach(
    P3_RPWM,
    PWM_FREQ,
    PWM_RES
  );
  ledcAttach(
    P3_LPWM,
    PWM_FREQ,
    PWM_RES
  );
  // ============================================================
  // STOP ALL PUMPS AT POWER-UP
  // ============================================================
  stopAllPumps();
  // ============================================================
  // FT-01 INTERRUPT
  // ============================================================
  attachInterrupt(
    digitalPinToInterrupt(FLOW_PIN),
    flowPulse,
    FALLING
  );
  // ============================================================
  // INITIAL SYSTEM STATE
  // ============================================================
  systemMode =
    MODE_OFF;
  manualStopLockout =
    false;
  totalVolume =
    0.0;
  currentFlowRate =
    0.0;
  excessFlowEvent =
    false;
  excessVolume =
    0.0;
  totalExcessVolume =
    0.0;
  case2ShutdownTriggered =
    false;
  // ============================================================
  // INITIAL FLOAT / BUTTON STATES
  // ============================================================
  lastFloatTriggered =
    isFloatTriggered();
  lastButtonState =
    digitalRead(
      ONOFF_PIN
    );
  stableButtonState =
    lastButtonState;
  lastDebounceTime =
    millis();
  previousFlowMillis =
    millis();
  // ============================================================
  // STARTUP INFORMATION
  // ============================================================
  Serial.println(
    "-----------------------------------------------"
  );
  Serial.println(
    "SYSTEM INITIALIZED"
  );
  Serial.println(
    "-----------------------------------------------"
  );
  Serial.println(
    "P1 : GPIO 33/32, EN 26/13"
  );
  Serial.println(
    "P2 : GPIO 19/18, EN 17/16"
  );
  Serial.println(
    "P3 : GPIO 25/23, EN 22/21"
  );
  Serial.println(
    "FLOAT : GPIO 35"
  );
  Serial.println(
    "FT-01 : GPIO 14"
  );
  Serial.println(
    "START : GPIO 34"
  );
  Serial.println(
    "SOLENOID : GPIO 27"
  );
  Serial.println(
    "-----------------------------------------------"
  );
  Serial.println(
    "POWER-UP SOLENOID = CLOSED"
  );
  Serial.println(
    "P1 = 50%"
  );
  Serial.println(
    "P2 = 100%"
  );
  Serial.println(
```

```
    "P3 = 100%"
  );
  Serial.println(
    "------------------------------------------------"
  );
  // ============================================================
  // CALIBRATION INFORMATION
  // ============================================================
  Serial.println(
    "FT-01 calibration:"
  );
  Serial.print(
    "a = "
  );
  Serial.println(
    CAL_A,
    8
  );
  Serial.print(
    "b = "
  );
  Serial.println(
    CAL_B,
    7
  );
  Serial.println(
    "------------------------------------------------"
  );
  Serial.print(
    "CASE 2 startup buffer = "
  );
  Serial.print(
    CASE2_STARTUP_BUFFER_MS / 1000
  );
  Serial.println(
    " seconds"
  );
  Serial.print(
    "CASE 2 shutdown threshold = "
  );
  Serial.print(
    CASE2_LOW_FLOW_THRESHOLD,
    2
  );
  Serial.println(
    " L/min"
  );
  Serial.println(
    "------------------------------------------------"
  );
  );
  Serial.println(
    "SYSTEM READY"
  );
  Serial.println();
  Serial.println(
    "Press physical momentary button"
  );
  Serial.println(
    "to start CASE 1."
  );
  Serial.println();
  Serial.println(
    "Type HELP for commands."
  );
  Serial.println(
    "================================================"
  );
}
// ============================================================
// MAIN LOOP
// ============================================================
void loop()
{
  // Physical button
  checkStartButton();
  // Serial commands
  processSerialCommand();
  // FT-01 measurement
  updateFlowMeasurement();
  // Case 1 float logic
  if (
    systemMode ==
    MODE_CASE1
  )
  {
    updateCase1FloatControl();
  }
  // Case 2 float logic
  if (
    systemMode ==
    MODE_CASE2
  )
  {
    updateCase2FloatControl();
  }
  // Ball valve measurement
  updateValveMeasurement();
}
```

3. Flow-Rate and Volume Calculation

The mean nominal flow rate over the complete Case 1 + Case 2 run is obtained from the ESP32 time stamps and the cumulative volume recorded in `samples.csv` (first and last DATA rows).

Step	Calculation	Result
1	ESP start time (first DATA row)	310906 ms
2	ESP stop time (last DATA row)	478453 ms
3	$\Delta t = 478453 - 310906$	167547 ms
4	167547 / 1000	167.547 s
5	167.547 / 60	2.79245 min
6	Total volume transferred (recorded, last row)	8.375004 L
7	$\bar{Q} = 8.375004 / 2.79245$	2.9991598 L/min

> **Mean nominal flow rate = 2.999 L/min (≈ 3.0 L/min)** over 167.547 s, with 8.375 L transferred.

As a cross-check, the arithmetic mean of the 167 individual flow samples in `samples.csv` is 3.0014 L/min, within 0.1 % of the value above. Both figures are FT-01-derived (calibrated) quantities, not an independent reference measurement.

4. Case 1 and Case 2 – Python Logging and Parsing Script

Script: `case1_case2_logger.py` (requires `pyserial`, `pandas`, `matplotlib`, `python-docx`). It opens the ESP32 COM port, forwards operator commands typed in the terminal, parses every incoming line with regular expressions, writes the data to CSV and builds the plots and the Word report when the run ends.

4.1 Parsed serial lines

Line	Format
`DATA`	`DATA,esp_millis,mode,pulses,frequency_hz,flow_lpm,total_volume_l,float_triggered,excess_event,solenoid`
`EVENT`	`EVENT,FLOAT_TRIGGER,<esp_millis>` and `EVENT,FLOAT_NORMAL,<esp_millis>` – used to compute float-event duration a sample-integrated event volume

4.2 Outputs written to each run folder

File	Content
`serial_log.txt`	Raw serial stream with host timestamps and the commands sent
`samples.csv`	All parsed DATA rows plus host timestamp
`events.csv`, `float_events.csv`	Raw EVENT lines; paired float trigger/return events
`case1_…png`, `case2_…png`, `ft01_flow_vs_frequency.png`	Flow vs time, cumulative volume vs time, flow vs frequency
`Case1_Case2_Experiment_Report.docx`	Auto-generated statistics, event table and plots

4.3 Command-line options

Option	Meaning
`--port`	Serial port of the ESP32 (required), e.g. COM5
`--baud`	Baud rate, default 115200
`--outdir`	Parent folder for run folders, default `case1_case2_runs`
`--a`, `--b`	Calibration coefficients used in the report (defaults 0.02420192 and 0.0259948)
`--calibration`	Optional CSV of timed collected-volume trials (pulses, duration_s, collected_volume_l) for independent calibration checking

4.4 Complete listing

```
#!/usr/bin/env python3
"""Separate Case 1 + Case 2 logger, plots, CSVs, and Word report.

Install: python -m pip install pyserial pandas matplotlib python-docx
Run:     python case1_case2_logger.py --port COM5
Commands typed in this terminal are sent to the ESP32: CASE1, CASE2, STOP,
STATUS, REPORT, RESET, HELP, /quit. Close Arduino Serial Monitor first.

The firmware DATA flow and cumulative volume are FT-01-calibrated estimates,
not independent reference measurements. Use --calibration for timed-volume
reference trials. Event volume is integrated from sampled FT-01 flow and is
also an estimate.
"""
from __future__ import annotations
import argparse, csv, datetime as dt, math, queue, re, sys, threading, time
from pathlib import Path
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import serial
from docx import Document
from docx.shared import Inches, Pt
```

```
DEFAULT_A, DEFAULT_B = 0.02420192, 0.02599480
DATA_RE = re.compile(r"^DATA,(\d+),([^,]+),(\d+),([-+0-9.eE]+),([-+0-9.eE]+),([-+0-9.eE]+),([01]),([01]),([01])\s*$")
EVENT_RE = re.compile(r"^EVENT,([^,]+),(\d+)\s*$")
COLS =
["esp_millis","mode","pulses","frequency_hz","flow_lpm","total_volume_l","float_triggered","excess_event","solenoid_open","host_timestamp"
]
def now():
    return dt.datetime.now().astimezone().isoformat(timespec="milliseconds")
def make_plots(df, out):
    paths=[]
    if df.empty: return paths
    df=df.copy()
    ts=pd.to_datetime(df.host_timestamp, errors="coerce")
    df["elapsed_s"]=(ts-ts.iloc[0]).dt.total_seconds()
    for mode in ("CASE1","CASE2"):
        s=df[df.mode==mode]
        if s.empty: continue
        fig,ax=plt.subplots(figsize=(10,5))
        ax.plot(s.elapsed_s,s.flow_lpm,linewidth=1)
        ax.set(title=f"{mode}: Flow Rate vs Time",xlabel="Elapsed time (s)",ylabel="Flow rate (L/min)")
        ax.grid(True,alpha=.3); fig.tight_layout()
        p=out/f"{mode.lower()}_flow_rate_vs_time.png"; fig.savefig(p,dpi=180); plt.close(fig); paths.append(p)
        fig,ax=plt.subplots(figsize=(10,5))
        ax.plot(s.elapsed_s,s.total_volume_l,linewidth=1)
        ax.set(title=f"{mode}: Cumulative FT-derived Volume vs Time",xlabel="Elapsed time (s)",ylabel="Cumulative volume (L)")
        ax.grid(True,alpha=.3); fig.tight_layout()
        p=out/f"{mode.lower()}_cumulative_volume.png"; fig.savefig(p,dpi=180); plt.close(fig); paths.append(p)
    fig,ax=plt.subplots(figsize=(8,5))
    ax.scatter(df.frequency_hz,df.flow_lpm,s=16,alpha=.7)
    ax.set(title="FT-01 Flow vs Pulse Frequency",xlabel="Pulse frequency (Hz)",ylabel="Firmware-calculated flow (L/min)")
    ax.grid(True,alpha=.3); fig.tight_layout()
    p=out/"ft01_flow_vs_frequency.png"; fig.savefig(p,dpi=180); plt.close(fig); paths.append(p)
    return paths
def build_report(out, df, events, a, b, cal_df, raw_path, plot_paths):
    doc=Document(); sec=doc.sections[0]; sec.top_margin=Inches(.65); sec.bottom_margin=Inches(.65)
    doc.styles["Normal"].font.name="Arial"; doc.styles["Normal"].font.size=Pt(9)
    doc.add_heading("LAM Stage 2 — Case 1 and Case 2 Report",0)
    doc.add_paragraph(f"Generated: {now()}")
    doc.add_paragraph(f"Calibration equation: Q = {a:.8f} f + {b:.7f}; Q in L/min, f in Hz.")
    doc.add_paragraph("Data note: flow and cumulative volume are derived from FT-01 and are not independent reference measurements.")
    doc.add_heading("1. Overall statistics",1)
    if df.empty: doc.add_paragraph("No DATA telemetry rows were captured. Confirm updated firmware, baud 115200, and correct COM port.")
    else:
        doc.add_paragraph(f"Samples: {len(df)}; mean flow {df.flow_lpm.mean():.3f} L/min; minimum {df.flow_lpm.min():.3f}; maximum
{df.flow_lpm.max():.3f}.")
        for mode in ("CASE1","CASE2"):
            s=df[df.mode==mode]
            if not s.empty:
                doc.add_heading(mode,2)
                doc.add_paragraph(f"Samples: {len(s)}; mean {s.flow_lpm.mean():.3f} L/min; min {s.flow_lpm.min():.3f}; max
{s.flow_lpm.max():.3f} L/min; last FT-derived total {s.total_volume_l.iloc[-1]:.4f} L.")
                doc.add_paragraph(f"Mean frequency {s.frequency_hz.mean():.3f} Hz; mean pulse count per DATA interval
{s.pulses.mean():.2f}.")
    doc.add_heading("2. Data fields recorded",1)
    doc.add_paragraph("esp_millis, mode, pulses, frequency_hz, flow_lpm, total_volume_l, float_triggered, excess_event, solenoid_open,
host_timestamp.")
    doc.add_heading("3. FT-01 frequency-to-flow relationship",1)
    doc.add_paragraph(f"The firmware reports Q = {a:.8f} × f + {b:.7f}. The flow-vs-frequency plot uses flow already calculated by the
firmware, so it visualizes the operating relationship but is not independent calibration validation.")
    doc.add_heading("4. Float event duration and estimated volume",1)
    if events:
        for i,e in enumerate(events,1):
            if e.get("start_s") is not None and e.get("end_s") is not None:
                duration=e["end_s"]-e["start_s"]
                vol=0.0
                s=df.sort_values("esp_millis")
                tm=s.esp_millis.to_numpy(float)/1000; fl=s.flow_lpm.to_numpy(float)
                for j in range(1,len(tm)):
                    overlap=max(0,min(tm[j],e["end_s"])-max(tm[j-1],e["start_s"]))
                    if overlap>0 and math.isfinite(fl[j]): vol+=max(0,fl[j])*overlap/60
                doc.add_paragraph(f"Event {i} ({e['type']} to FLOAT_NORMAL): duration {duration:.3f} s; FT-sample-integrated volume
estimate {vol:.4f} L.")
    else: doc.add_paragraph("No complete FLOAT_TRIGGER to FLOAT_NORMAL event was captured.")
    doc.add_paragraph("Event volume is a sampled FT-derived estimate. Compare against independently measured tank/collected volume over
the same event boundaries.")
    if cal_df is not None:
        doc.add_heading("5. Independent collected-volume calibration",1)
        x=cal_df.frequency_hz.to_numpy(float); y=cal_df.independent_flow_lpm.to_numpy(float)
        pred=a*x+b; err=pred-y
        ssres=float((err**2).sum()); sstot=float(((y-y.mean())**2).sum())
        r2=1-ssres/sstot if sstot>1e-12 else float("nan")
        doc.add_paragraph(f"Trials: {len(cal_df)}; fixed-equation R²={r2:.5f}; MAE={abs(err).mean():.4f} L/min; maximum absolute
error={abs(err).max():.4f} L/min.")
        c=doc.add_table(rows=1,cols=5); c.style="Light Shading Accent 1"
        for cell,label in zip(c.rows[0].cells,["Trial","f (Hz)","Reference Q","Predicted Q","Error"]): cell.text=label
        for i,row in cal_df.iterrows():
            cells=c.add_row().cells

vals=[str(row.get("trial",i+1)),f"{row.frequency_hz:.4f}",f"{row.independent_flow_lpm:.4f}",f"{(a*row.frequency_hz+b):.4f}",f"{(a*row.freq
uency_hz+b-row.independent_flow_lpm):+.4f}"]
            for cell,val in zip(cells,vals): cell.text=val
    doc.add_heading("6. Plots generated",1)
    for p in plot_paths:
        doc.add_paragraph(p.name)
        doc.add_picture(str(p),width=Inches(6.2))
    doc.add_heading("7. Limitations",1)
```

```
    doc.add_paragraph("The logger cannot independently measure water volume. Calibrate using timed collected-volume trials. The script
does not alter firmware coefficients. Verify GPIO wiring, pump-driver enable logic, float-switch polarity, solenoid fail-state, and
emergency stop before operation.")
    path=out/"Case1_Case2_Experiment_Report.docx"; doc.save(path); return path

def load_cal(path):
    if not path: return None
    d=pd.read_csv(path)
    need={"pulses","duration_s","collected_volume_l"}
    if not need.issubset(d.columns): raise ValueError(f"Calibration CSV needs columns: {sorted(need)}")
    for c in need: d[c]=pd.to_numeric(d[c],errors="coerce")
    d["frequency_hz"]=pd.to_numeric(d["frequency_hz"],errors="coerce") if "frequency_hz" in d else float("nan")
    m=d.frequency_hz.isna()
    d.loc[m,"frequency_hz"]=d.loc[m,"pulses"]/d.loc[m,"duration_s"]
    d["independent_flow_lpm"]=60*d.collected_volume_l/d.duration_s
    d=d.replace([float("inf"),-
float("inf")],float("nan")).dropna(subset=["frequency_hz","duration_s","collected_volume_l","independent_flow_lpm"])
    d=d[(d.duration_s>0)&(d.collected_volume_l>=0)]
    if len(d)<2: raise ValueError("At least two valid calibration rows are required.")
    return d

def main():
    ap=argparse.ArgumentParser(description="Separate Case 1 + Case 2 ESP32 logger and report")
    ap.add_argument("--port",required=True); ap.add_argument("--baud",type=int,default=115200)
    ap.add_argument("--outdir",default="case1_case2_runs"); ap.add_argument("--a",type=float,default=DEFAULT_A); ap.add_argument("--
b",type=float,default=DEFAULT_B)
    ap.add_argument("--calibration",default=""); args=ap.parse_args()
    out=Path(args.outdir)/dt.datetime.now().strftime("run_%Y%m%d_%H%M%S"); out.mkdir(parents=True)
    raw_path=out/"serial_log.txt"; sample_path=out/"samples.csv"; event_path=out/"events.csv"
    cal=load_cal(args.calibration) if args.calibration else None
    rows=[]; events=[]; raw=[]; open_event=None
    print(f"Output folder: {out.resolve()}")
    print("Type CASE1, CASE2, STOP, STATUS, REPORT, RESET, HELP. Type /quit to finish. Close Arduino Serial Monitor first.")
    ser=serial.Serial(args.port,args.baud,timeout=.25); time.sleep(2)
    q=queue.Queue(); stop=threading.Event()
    def keyboard():
        while not stop.is_set():
            line=sys.stdin.readline()
            if not line: break
            line=line.strip()
            if line.lower()=="/quit": stop.set(); break
            if line: q.put(line)
    threading.Thread(target=keyboard,daemon=True).start()
    with raw_path.open("w",encoding="utf-8",buffering=1) as rf, sample_path.open("w",newline="",encoding="utf-8") as sf,
event_path.open("w",newline="",encoding="utf-8") as ef:
        sw=csv.DictWriter(sf,fieldnames=COLS); sw.writeheader()
        ew=csv.DictWriter(ef,fieldnames=["host_timestamp","event","esp_millis"]); ew.writeheader()
        try:
            while not stop.is_set():
                while not q.empty():
                    cmd=q.get(); ser.write((cmd+"\n").encode()); rf.write(f"[HOST {now()}] {cmd}\n"); print(f"[sent] {cmd}")
                b=ser.readline()
                if not b: continue
                line=b.decode("utf-8",errors="replace").strip(); ts=now()
                print(line); rf.write(f"[{ts}] {line}\n"); raw.append(line)
                m=DATA_RE.match(line)
                if m:

row={"esp_millis":int(m.group(1)),"mode":m.group(2),"pulses":int(m.group(3)),"frequency_hz":float(m.group(4)),"flow_lpm":float(m.group(5))
,"total_volume_l":float(m.group(6)),"float_triggered":int(m.group(7)),"excess_event":int(m.group(8)),"solenoid_open":int(m.group(9)),"host
_timestamp":ts}
                    rows.append(row); sw.writerow(row); sf.flush()
                e=EVENT_RE.match(line)
                if e:
                    name,ms=e.group(1),int(e.group(2)); ew.writerow({"host_timestamp":ts,"event":name,"esp_millis":ms}); ef.flush()
                    if name=="FLOAT_TRIGGER": open_event={"type":name,"start_s":ms/1000,"esp_millis":ms}
                    elif name=="FLOAT_NORMAL" and open_event:
                        open_event["end_s"]=ms/1000; events.append(open_event); open_event=None
                if "SYSTEM STOP COMPLETE" in line: time.sleep(.5); stop.set()
        except KeyboardInterrupt: print("\nCtrl+C: finalizing.")
        finally: stop.set(); ser.close()
    df=pd.DataFrame(rows,columns=COLS)
    if not df.empty: df.to_csv(sample_path,index=False)
    pd.DataFrame(events).to_csv(out/"float_events.csv",index=False)
    plots=make_plots(df,out); report=build_report(out,df,events,args.a,args.b,cal,raw_path,plots)
    print("\nFINALIZED"); print(f"Samples: {sample_path}\nEvents: {event_path}\nReport: {report}\nPlots: {len(plots)}")
if __name__=="__main__": main()
```

5. Case 1 and Case 2 – Final Results and Graphs

5.1 Summary statistics (from samples.csv)

Phase	Samples	Duration (s)	Mean (L/min)	Min (L/min)	Max (L/min)	Cumulative volume (L)
Case 1	52	51.0	2.991	1.525	3.075	2.5966
Case 2	115	115.0	3.006	0.316	5.471	8.3750 (run total)
Overall	167	167.5	3.001	0.316	5.471	8.3750

Volume is cumulative across the two phases: Case 1 contributed 2.5966 L and Case 2 a further 5.7785 L. The Case 1 minimum (1.525 L/min) is the first, partial sampling window after start-up.

5.2 Flow rate vs time

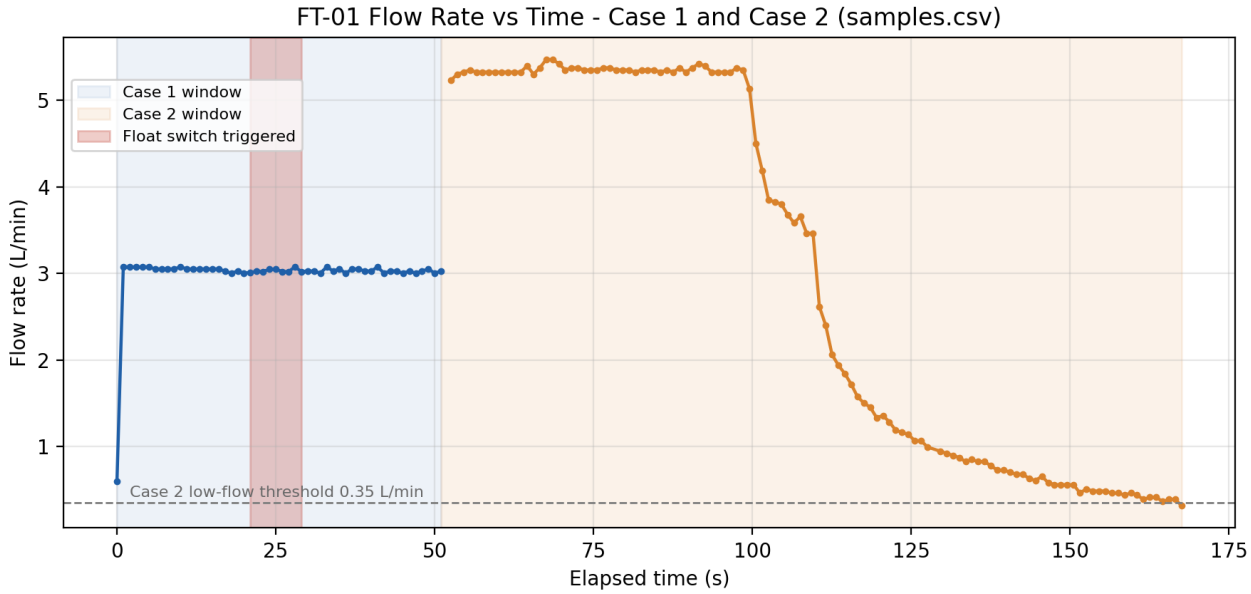


Figure 5.1 – FT-01 flow rate during the Case 1 → Case 2 run (samples.csv)

- **Case 1:** flow settles at about 3.0 L/min (maximum 3.075 L/min) within the first second and stays there.
- **Float event:** nine consecutive samples carry `float_triggered = 1` and `excess_event = 1` with the solenoid closed; flow stays at about 3.03 L/min, showing that the pumps continued while the solenoid was closed, as designed.
- **Case 2:** with all three pumps running the flow rises to about 5.3–5.5 L/min (maximum 5.471 L/min), then decays as the flush tank empties. The last sample (0.316 L/min) is below the 0.35 L/min threshold, which ends the run through the automatic low-flow shutdown.

5.3 Cumulative volume

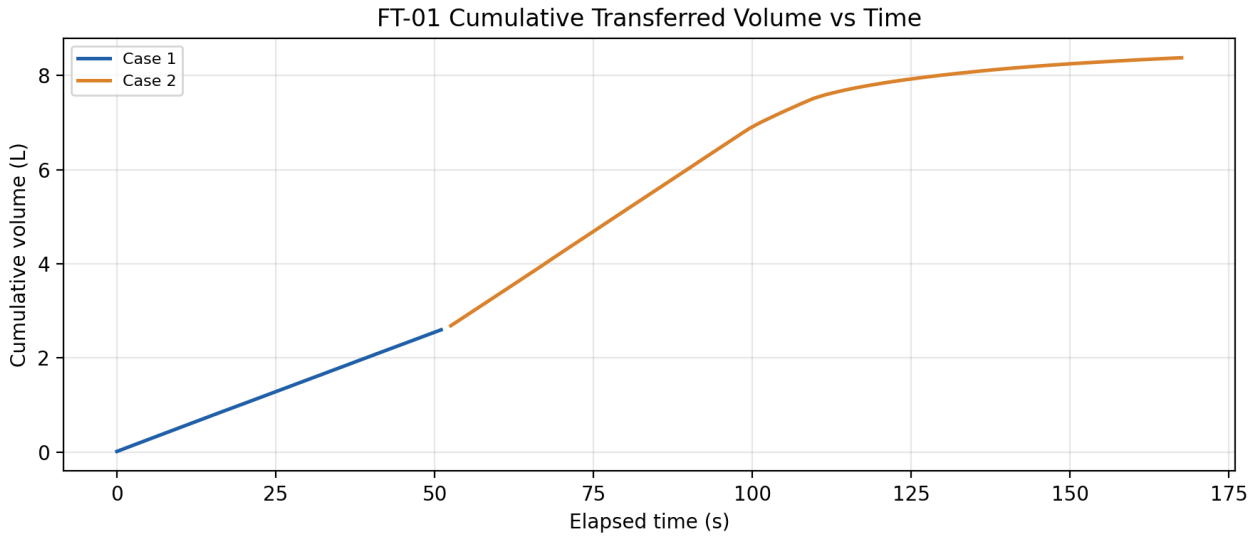


Figure 5.2 – FT-01-derived cumulative volume (8.375 L at end of run)

5.4 Cumulative volume (case1)

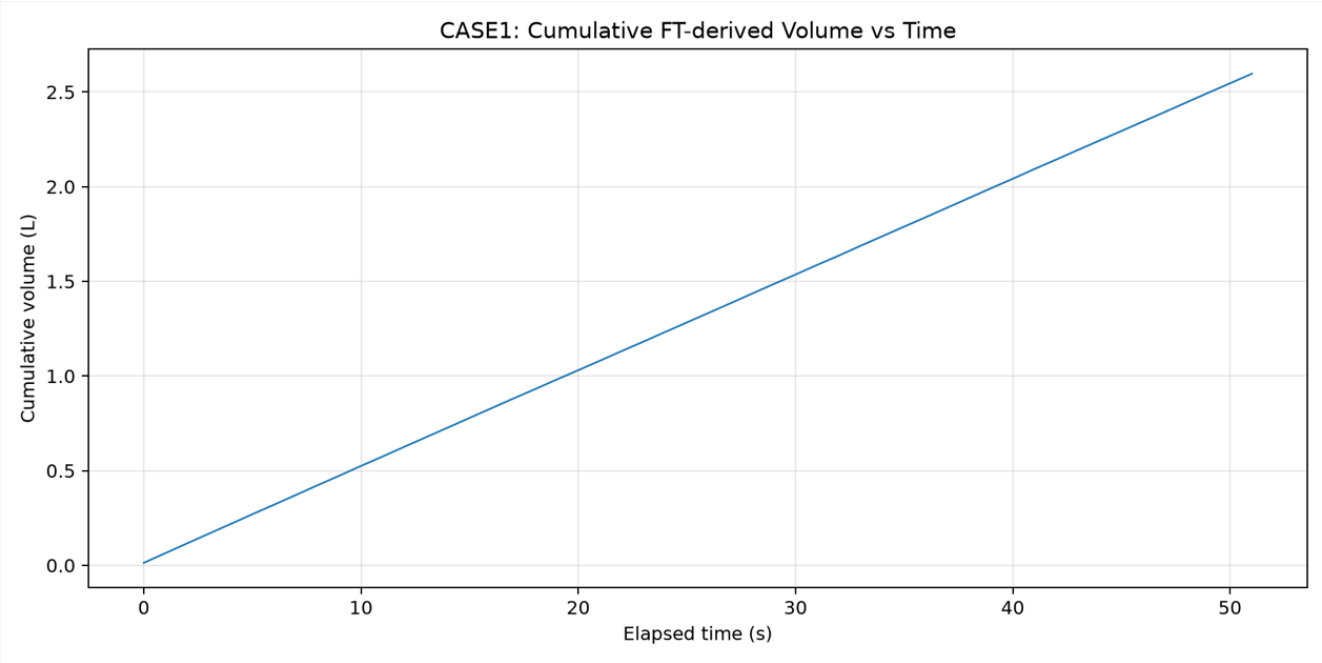


Figure 5.3 – FT-01-derived cumulative volume for case1

5.5 Cumulative volume (case2)

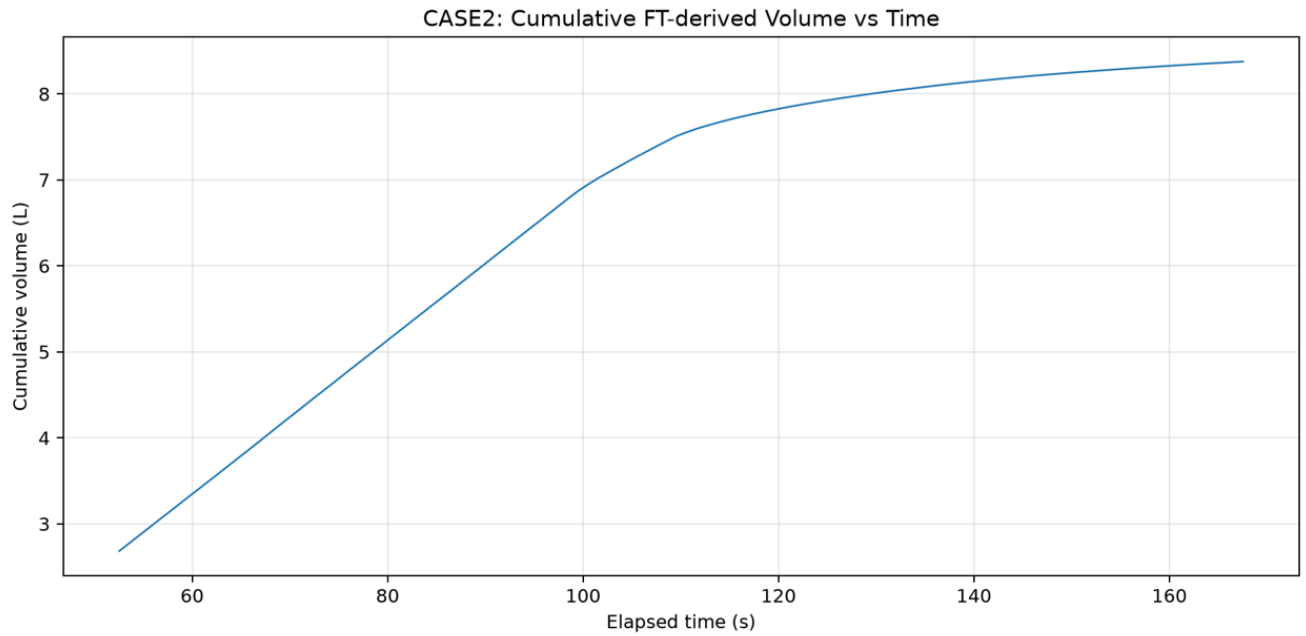


Figure 5.4 – FT-01-derived cumulative volume for case2

5.6 Flow rate vs time (Case1)

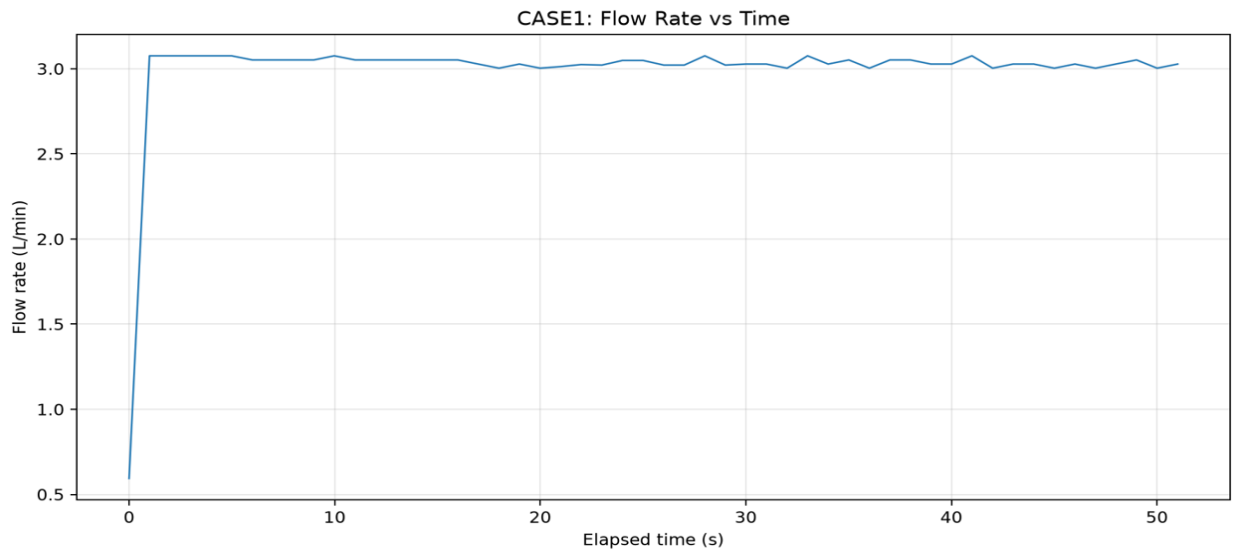


Figure 5.5 – FT-01 flow rate during the Case 1

5.7 Flow rate vs time (Case2)

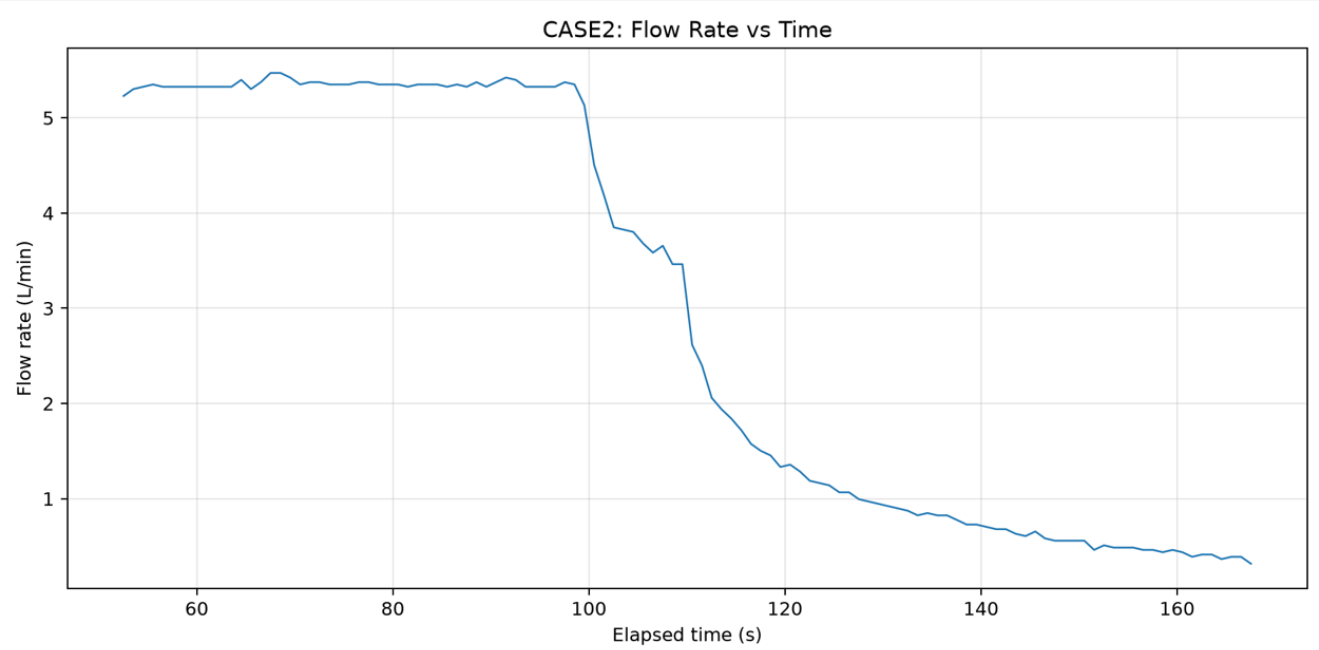


Figure 5.6 – FT-01 flow rate during the Case 2

5.8 Float-switch event

Quantity	Value
Float event duration (EVENT,FLOAT_TRIGGER → FLOAT_NORMAL)	8.727 s
Sample-integrated event volume (FT-01 estimate)	0.4412 L
Samples flagged `excess_event = 1` / mean flow in them	9 samples / 3.032 L/min

Event duration and volume come from the logger report of this run; the sample count and mean flow are from `samples.csv`. The volume is an FT-01-derived estimate.

5.9 Integrated run plot

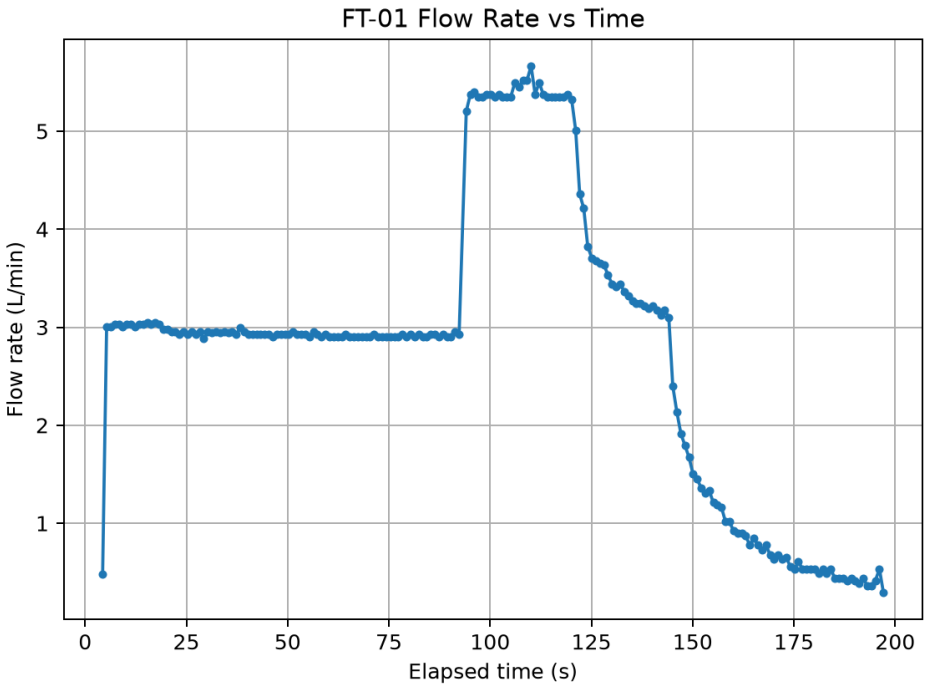


Figure 5.7 – FT-01 flow rate vs elapsed time, integrated plot of a complete Case 1 → Case 2 run

This plot shows the same behaviour as Figure 5.1: a Case 1 plateau near 3 L/min, a step to about 5.4 L/min when Case 2 starts, then a gradual decay towards 0.3 L/min as the flush tank empties.

> **Data note:** flow and volume are calibrated FT-01 estimates, not independent reference measurements.

6. Ball Valve Position vs Flow Rate – Experiment Overview

This experiment characterises how the flow rate measured by FT-01 depends on the opening of the manual ball valve. Four positions are tested – 25 %, 50 %, 75 % and 100 % open. The valve opening is a physical, manually set position; it is **not** a pump PWM duty cycle.

6.1 Procedure

Step	Action
1	Start the Python ball-valve logger and the standalone ESP32 sketch (Serial Monitor closed).
2	Type `VALVE`. The ESP32 asks for the valve position.
3	Physically set the ball valve to 25, 50, 75 or 100 % and confirm by typing that number.
4	The ESP32 waits 4 s for the flow to stabilise, then averages the flow for 5 s.
5	The ESP32 prints `Stable flow = … L/min`; the Python script records it against the valve opening.
6	Repeat for the next position; `/quit` finalises the run and writes the report.

6.2 Key points

- The standalone sketch only counts FT-01 pulses and reports flow and volume; it does not move the valve and does not control the pumps.
- Flow uses the same FT-01 calibration, $Q = 0.02420192 \cdot f + 0.0259948$ L/min.
- Repeating each position several times gives a mean and standard deviation per opening.

7. Ball Valve Position vs Flow Rate – ESP32 Code

Firmware file: `Standalone_Ball_Valve_Flow_Characterization_Calibrated.ino`. Commands: `HELP`, `VALVE`, `25`, `50`, `75`, `100`, `STATUS`, `REPORT`, `RESET`, `STOP`. A `DATA` line is printed every second and, after each averaging period, `Stable flow = … L/min` and `VALVE_RESULT,<opening>,<flow>,<samples>` are printed.

7.1 Complete listing

```
/*
  LAM Stage 2 - Standalone Ball-Valve Flow Characterization
  Serial baud: 115200
  Commands: HELP, VALVE, 25, 50, 75, 100, STATUS, REPORT, RESET, STOP

  Safety:
  - This sketch only counts FT-01 pulses and reports flow/volume.
  - It does NOT automatically move a manual ball valve.
  - Operator must physically set valve opening before confirming position.
  - Pump outputs are held LOW; use the rig's independently verified safe pump
    operating arrangement. This standalone sketch intentionally does not start pumps.
  - Verify the actual wiring/pin map before connecting any hardware.
*/
#include <Arduino.h>
#define FLOW_PIN 14
#define FLOAT_PIN 35
#define ONOFF_PIN 34
#define SOLENOID_PIN 27

// FT-01 calibration: Q (L/min) = a * frequency (Hz) + b
const float CAL_A = 0.02420192f;
const float CAL_B = 0.02599480f;

const unsigned long SAMPLE_MS = 1000;
const unsigned long VALVE_STABILIZATION_MS = 4000;
const unsigned long VALVE_AVERAGING_MS = 5000;
volatile unsigned long pulseCount = 0;
unsigned long previousSample = 0;
unsigned long totalPulses = 0;
float flowLpm = 0.0f, totalVolumeL = 0.0f;
bool valvePrompt = false, measuring = false;
int selectedPosition = 0;
unsigned long stabilizationStart = 0, averagingStart = 0;
float flowSum = 0.0f; unsigned long flowSamples = 0;
String commandBuffer;

void IRAM_ATTR flowPulse() { pulseCount++; }

void printCalibration() {
  Serial.println(F("FT-01 calibration (Q = a*f + b):"));
  Serial.print(F("a = "));
  Serial.println(CAL_A, 8);
  Serial.print(F("b = "));
  Serial.println(CAL_B, 8);
}

void printStatus() {
  Serial.println(F("=== BALL VALVE TEST STATUS ==="));
  Serial.print(F("Valve test active: ")); Serial.println(measuring ? F("YES") : F("NO"));
  Serial.print(F("Selected opening: ")); Serial.print(selectedPosition); Serial.println(F("%"));
  Serial.print(F("Frequency Hz: ")); Serial.println((float)totalPulses / (SAMPLE_MS / 1000.0f), 3);
  Serial.print(F("Flow L/min: ")); Serial.println(flowLpm, 3);
  Serial.print(F("FT-derived total volume L: ")); Serial.println(totalVolumeL, 4);
  printCalibration();
}
void printHelp() {
  Serial.println(F("Commands: VALVE, 25, 50, 75, 100, STATUS, REPORT, RESET, STOP, HELP"));
  Serial.println(F("Set the physical valve position first; this sketch does not actuate a manual valve."));
}
void beginPosition(int pos) {
  if (!(pos == 25 || pos == 50 || pos == 75 || pos == 100)) {
    Serial.println(F("Invalid valve position. Use 25, 50, 75, or 100."));
    return;
  }
  selectedPosition = pos;
  valvePrompt = false;
  measuring = false;
  flowSum = 0; flowSamples = 0;
  stabilizationStart = millis();
  Serial.print(F("BALL VALVE POSITION = ")); Serial.print(pos); Serial.println(F("%"));
  Serial.println(F("Stabilizing flow for 4 seconds; keep valve/pump arrangement unchanged."));
}
void handleCommand(String cmd) {
  cmd.trim(); cmd.toUpperCase();
  if (cmd == "HELP") printHelp();
  else if (cmd == "VALVE") {
    valvePrompt = true;
    Serial.println(F("BALL VALVE FLOW MEASUREMENT"));
    Serial.println(F("Physically set valve opening, then enter 25, 50, 75, or 100."));
  } else if (cmd == "25" || cmd == "50" || cmd == "75" || cmd == "100") {
    if (!valvePrompt) Serial.println(F("Type VALVE first, then enter the physical valve position."));
    else beginPosition(cmd.toInt());
  } else if (cmd == "STATUS" || cmd == "REPORT") printStatus();
  else if (cmd == "RESET") { noInterrupts(); pulseCount=0; interrupts(); totalPulses=0; totalVolumeL=0; Serial.println(F("TOTALS RESET"));
}
  else if (cmd == "STOP") { measuring=false; valvePrompt=false; Serial.println(F("VALVE MEASUREMENT STOPPED; pump outputs were not
controlled by this sketch.")); }
```

```
  else if (cmd.length()) Serial.println(F("Unknown command. Type HELP."));
}
void setup() {
  Serial.begin(115200);
  pinMode(FLOW_PIN, INPUT);
  pinMode(FLOAT_PIN, INPUT);
  pinMode(ONOFF_PIN, INPUT);
  pinMode(SOLENOID_PIN, OUTPUT);
  digitalWrite(SOLENOID_PIN, LOW); // verify this is the safe closed state for your hardware
  attachInterrupt(digitalPinToInterrupt(FLOW_PIN), flowPulse, RISING);
  previousSample=millis();
  Serial.println(F("LAM Stage 2 - Standalone Ball-Valve Flow Logger"));
  Serial.println(F("Pump control intentionally not implemented in this sketch."));
  printCalibration();
  printHelp();
}
void loop() {
  while (Serial.available()) {
    char c=(char)Serial.read();
    if (c=='\n' || c=='\r') { if (commandBuffer.length()) { handleCommand(commandBuffer); commandBuffer=""; } }
    else if (commandBuffer.length()<80) commandBuffer += c;
  }
  unsigned long nowMs=millis();
  if (nowMs-previousSample >= SAMPLE_MS) {
    unsigned long elapsed=nowMs-previousSample; previousSample=nowMs;
    noInterrupts(); unsigned long pulses=pulseCount; pulseCount=0; interrupts();
    totalPulses += pulses;
    float frequency=(elapsed>0) ? pulses*1000.0f/elapsed : 0;
    flowLpm=CAL_A*frequency+CAL_B;
    if (flowLpm<0) flowLpm=0;
    totalVolumeL += flowLpm*((float)elapsed/60000.0f);
    Serial.print(F("DATA,")); Serial.print(nowMs); Serial.print(F(",VALVE,"));
    Serial.print(pulses); Serial.print(','); Serial.print(frequency,3); Serial.print(',');
    Serial.print(flowLpm,4); Serial.print(','); Serial.print(totalVolumeL,5); Serial.print(',');
    Serial.print(digitalRead(FLOAT_PIN)==HIGH ? 1:0); Serial.print(F(",0,0\n"));
  }
  if (!valvePrompt && selectedPosition>0 && !measuring && nowMs-stabilizationStart >= VALVE_STABILIZATION_MS) {
    measuring=true; averagingStart=nowMs; flowSum=0; flowSamples=0;
    Serial.println(F("Flow stabilization complete; averaging for 5 seconds."));
  }
  if (measuring) {
    flowSum += flowLpm; flowSamples++;
    if (nowMs-averagingStart >= VALVE_AVERAGING_MS) {
      float avg=flowSamples ? flowSum/flowSamples : 0;
      Serial.print(F("Stable flow = ")); Serial.print(avg,3); Serial.println(F(" L/min"));
      Serial.print(F("VALVE_RESULT,")); Serial.print(selectedPosition); Serial.print(','); Serial.print(avg,4); Serial.print(',');
Serial.println(flowSamples);
      measuring=false; selectedPosition=0;
      Serial.println(F("Type VALVE for the next position."));
    }
  }
}
```

8. Ball Valve Position vs Flow Rate – Python Script

Script: `ball_valve_flow_logger.py` (run with `python ball_valve_flow_logger.py --port COM5`). It forwards the commands to the ESP32, parses `DATA` lines, the `BALL VALVE POSITION = n%` line and the `Stable flow = x L/min` line, and links each stable-flow value to the valve opening that was active.

Output file	Content
`serial_log.txt`	Raw serial stream with host timestamps and commands sent
`samples.csv`	All parsed DATA rows
`valve_results.csv`	One row per stable-flow result: opening %, stable flow, timestamp
`valve_summary.csv`	Per-opening count, mean, SD, min, max
`ball_valve_opening_vs_flow.png`, `ft01_flow_vs_frequency.png`	Opening vs flow (mean ± SD) and flow vs frequency plots
`Ball_Valve_Experiment_Report.docx`	Auto-generated results table and plots

8.1 Complete listing

```
#!/usr/bin/env python3
"""Separate ball-valve opening vs flow logger/report.

Install: python -m pip install pyserial pandas matplotlib python-docx
Run: python ball_valve_flow_logger.py --port COM5
Commands are sent to ESP32: VALVE, 25, 50, 75, 100, STOP, STATUS, REPORT,
HELP, /quit. The operator must physically set the valve opening to the requested
position before confirming it. Do not treat valve opening percentage as pump duty.
"""
from __future__ import annotations
import argparse, csv, datetime as dt, math, queue, re, sys, threading, time
from pathlib import Path
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import serial
from docx import Document
from docx.shared import Inches, Pt

DATA_RE=re.compile(r"^DATA,(\d+),([^,]+),(\d+),([-+0-9.eE]+),([-+0-9.eE]+),([-+0-9.eE]+),([01]),([01]),([01])\s*$")
VALVE_RE=re.compile(r"BALL VALVE POSITION\s*=\s*(25|50|75|100)%",re.I)
FLOW_RE=re.compile(r"Stable flow\s*=\s*([-+0-9.eE]+)\s*L/min",re.I)
COLS=["esp_millis","mode","pulses","frequency_hz","flow_lpm","total_volume_l","float_triggered","excess_event","solenoid_open","host_timestamp"]

def now(): return dt.datetime.now().astimezone().isoformat(timespec="milliseconds")

def main():
    ap=argparse.ArgumentParser(description="Ball-valve experiment logger and report")
    ap.add_argument("--port",required=True); ap.add_argument("--baud",type=int,default=115200); ap.add_argument("--outdir",default="ball_valve_runs")
    ap.add_argument("--a",type=float,default=0.02420192); ap.add_argument("--b",type=float,default=0.02599480)
    args=ap.parse_args()
    out=Path(args.outdir)/dt.datetime.now().strftime("run_%Y%m%d_%H%M%S"); out.mkdir(parents=True)
    raw_path=out/"serial_log.txt"; samples_path=out/"samples.csv"; results_path=out/"valve_results.csv"
    rows=[]; raw=[]; results=[]; active_pos=None
    print(f"Output folder: {out.resolve()}")
    print("Commands: VALVE starts a valve test; physically set valve to prompted opening; enter 25, 50, 75, 100 when requested.")
    print("Type STOP to stop, REPORT for ESP32 report, /quit to finalize. Close Arduino Serial Monitor first.")
    ser=serial.Serial(args.port,args.baud,timeout=.25); time.sleep(2)
    q=queue.Queue(); stop=threading.Event()
    def keyboard():
        while not stop.is_set():
            line=sys.stdin.readline()
            if not line: break
            line=line.strip()
            if line.lower()=="/quit": stop.set(); break
            if line: q.put(line)
    threading.Thread(target=keyboard,daemon=True).start()
    with raw_path.open("w",encoding="utf-8",buffering=1) as rf, samples_path.open("w",newline="",encoding="utf-8") as sf:
        writer=csv.DictWriter(sf,fieldnames=COLS); writer.writeheader()
        try:
            while not stop.is_set():
                while not q.empty():
                    cmd=q.get(); ser.write((cmd+"\n").encode()); rf.write(f"[HOST {now()}] {cmd}\n"); print(f"[sent] {cmd}")
                b=ser.readline()
                if not b: continue
                line=b.decode("utf-8",errors="replace").strip(); ts=now(); print(line); rf.write(f"[{ts}] {line}\n"); raw.append(line)
                vm=VALVE_RE.search(line)
                if vm: active_pos=int(vm.group(1))
                fm=FLOW_RE.search(line)
                if fm and active_pos is not None:
```

```
                    results.append({"opening_percent":active_pos,"stable_flow_lpm":float(fm.group(1)),"source":"ESP32 stable-flow
result","host_timestamp":ts}); active_pos=None
                dm=DATA_RE.match(line)
                if dm:

row={"esp_millis":int(dm.group(1)),"mode":dm.group(2),"pulses":int(dm.group(3)),"frequency_hz":float(dm.group(4)),"flow_lpm":float(dm.grou
p(5)),"total_volume_l":float(dm.group(6)),"float_triggered":int(dm.group(7)),"excess_event":int(dm.group(8)),"solenoid_open":int(dm.group(
9)),"host_timestamp":ts}
                    rows.append(row); writer.writerow(row); sf.flush()
                if "SYSTEM STOP COMPLETE" in line: time.sleep(.5); stop.set()
        except KeyboardInterrupt: print("\nCtrl+C: finalizing.")
        finally: stop.set(); ser.close()
    pd.DataFrame(rows,columns=COLS).to_csv(samples_path,index=False)
    # Preserve actual stable-flow outputs and aggregate repeats by opening.
    r=pd.DataFrame(results,columns=["opening_percent","stable_flow_lpm","source","host_timestamp"])
    r.to_csv(results_path,index=False)
    plots=[]
    if not r.empty:
        summary=r.groupby("opening_percent").stable_flow_lpm.agg(["count","mean","std","min","max"]).reset_index()
        summary.to_csv(out/"valve_summary.csv",index=False)
        fig,ax=plt.subplots(figsize=(8,5))
        ax.errorbar(summary.opening_percent,summary["mean"],yerr=summary["std"].fillna(0),fmt="o-",capsize=4)
        ax.set(title="Ball-Valve Opening vs Measured Flow",xlabel="Valve opening (%)",ylabel="Stable flow (L/min)")
        ax.grid(True,alpha=.3); fig.tight_layout()
        p=out/"ball_valve_opening_vs_flow.png"; fig.savefig(p,dpi=180); plt.close(fig); plots.append(p)
    if not pd.DataFrame(rows).empty:
        d=pd.DataFrame(rows)
        fig,ax=plt.subplots(figsize=(8,5)); ax.scatter(d.frequency_hz,d.flow_lpm,s=16,alpha=.7)
        ax.set(title="FT-01 Flow vs Pulse Frequency During Valve Tests",xlabel="Pulse frequency (Hz)",ylabel="Firmware-calculated flow
(L/min)")
        ax.grid(True,alpha=.3); fig.tight_layout(); p=out/"ft01_flow_vs_frequency.png"; fig.savefig(p,dpi=180); plt.close(fig);
plots.append(p)
    doc=Document(); sec=doc.sections[0]; sec.top_margin=Inches(.65); sec.bottom_margin=Inches(.65)
    doc.styles["Normal"].font.name="Arial"; doc.styles["Normal"].font.size=Pt(9)
    doc.add_heading("LAM Stage 2 — Ball Valve Characterization Report",0); doc.add_paragraph(f"Generated: {now()}")
    doc.add_paragraph("Valve opening percentage is a manually set ball-valve position, not pump PWM duty cycle. Flow values are FT-01-
derived unless independently measured.")
    doc.add_heading("1. Stable flow by valve position",1)
    if r.empty: doc.add_paragraph("No recognised stable-flow valve results were recorded. Check firmware output includes BALL VALVE
POSITION and Stable flow lines.")
    else:
        doc.add_paragraph(f"Recognised measurements: {len(r)}; overall mean flow {r.stable_flow_lpm.mean():.3f} L/min; min
{r.stable_flow_lpm.min():.3f}; max {r.stable_flow_lpm.max():.3f}.")
        t=doc.add_table(rows=1,cols=6); t.style="Light Shading Accent 1"
        for c,h in zip(t.rows[0].cells,["Opening %","Trials","Mean L/min","SD L/min","Min L/min","Max L/min"]): c.text=h
        for _,row in summary.iterrows():
            cells=t.add_row().cells
            vals=[str(int(row.opening_percent)),str(int(row["count"])),f"{row['mean']:.3f}",f"{row['std']:.3f}" if pd.notna(row["std"])
else "N/A",f"{row['min']:.3f}",f"{row['max']:.3f}"]
            for c,v in zip(cells,vals): c.text=v
    doc.add_heading("2. Captured telemetry",1)
    d=pd.DataFrame(rows)
    if not d.empty: doc.add_paragraph(f"Samples: {len(d)}; mean flow {d.flow_lpm.mean():.3f} L/min; min {d.flow_lpm.min():.3f}; max
{d.flow_lpm.max():.3f} L/min; mean frequency {d.frequency_hz.mean():.3f} Hz.")
    else: doc.add_paragraph("No DATA telemetry rows captured.")
    doc.add_heading("3. Data integrity and limitations",1)
    doc.add_paragraph("The FT-01 frequency-to-flow plot is not independent calibration validation because the flow value comes from the
same sensor/calibration. For independent calibration, collect a known volume over a timed interval and compare reference flow with
frequency.")
    doc.add_paragraph("Repeat each valve position several times after the flow stabilizes. Mean and sample standard deviation are reported
for repeated trials. If only one trial exists, SD is undefined.")
    for p in plots: doc.add_paragraph(p.name); doc.add_picture(str(p),width=Inches(6.2))
    report=out/"Ball_Valve_Experiment_Report.docx"; doc.save(report)
    print("\nFINALIZED"); print(f"Raw log: {raw_path}\nSamples: {samples_path}\nValve results: {results_path}\nReport: {report}\nPlots:
{len(plots)}")
if __name__=="__main__": main()
```

9. Ball Valve Position vs Flow Rate – Results and Graphs

Valve opening (%)	Trials	Stable flow (L/min)	Min recorded (L/min)	Max recorded (L/min)
25	1	0.314	0.220	0.365
50	1	1.938	1.938	1.938
75	1	3.015	3.003	3.051
100	1	3.119	3.100	3.124

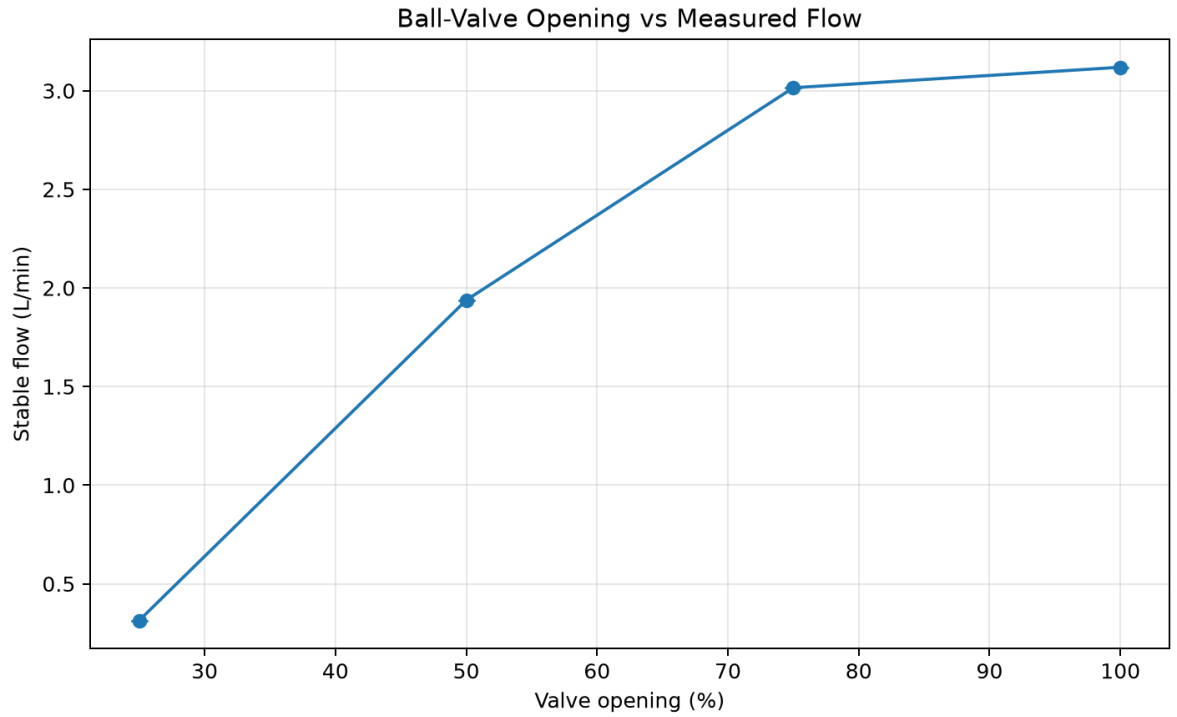


Figure 9.1 – Ball-valve opening vs measured stable flow (FT-01)

9.1 Observations

- Flow rises steeply from 25 % to 75 % opening (0.314 → 1.938 → 3.015 L/min).
- From 75 % to 100 % the gain is only 0.104 L/min (about 3.4 %), so beyond roughly 75 % the flow is limited mainly by the pump and the rest of the circuit rather than by the valve.
- Each position has a single trial, so a standard deviation cannot be computed yet. Repeating each opening several times will give mean ± SD and confirm repeatability.

Data note: flow values are FT-01-derived and are not independently measured.

10. Interfacing the ESP32 with the Python Script over the Serial Bus

The ESP32 and the PC communicate over the USB serial bus at 115200 baud. The Python logger runs in a command terminal, sends the operator's commands to the ESP32 and, at the same time, reads and parses everything the ESP32 prints.

10.1 Setup and commands

Step	Action
1	Upload the firmware to the ESP32 and close the Arduino Serial Monitor so the COM port is free.
2	Open a command terminal and start the logger, e.g. `python case1_case2_logger.py --port COM5` (or `python ball_valve_flow_logger.py --port COM5`).
3	Type commands in the same terminal: `CASE1`, `CASE2`, `STOP`, `STATUS`, `REPORT`, `RESET`, `HELP` (and `VALVE`, `25/50/75/100` for the valve test).
4	Type `/quit` (or let the ESP32 finish with its stop sequence) to finalise the run. CSV files, plots and the Word report are written to the run folder.

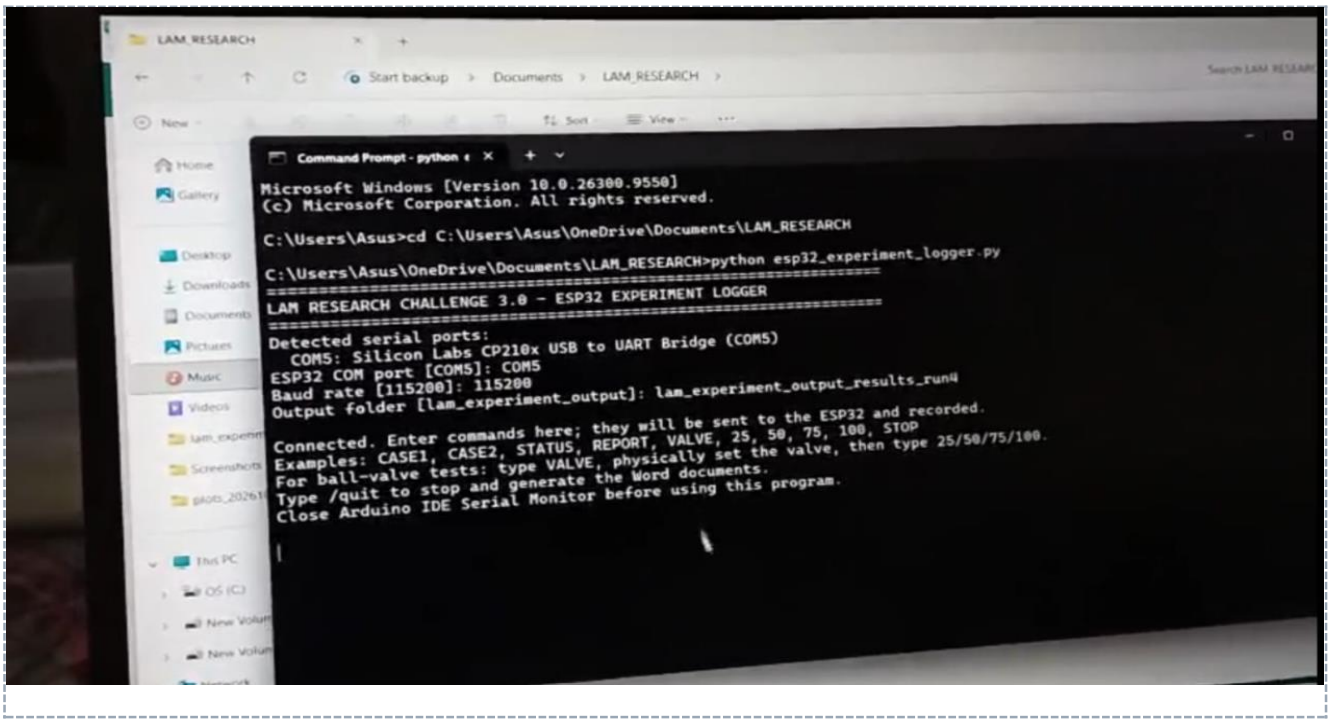


Figure 10.1 – ESP32 interfaced with the Python script through the serial bus in the command terminal

10.2 Verification runs

Before the final data set was recorded, several test runs were carried out to check that the system works correctly: that the serial link and the Python parsing capture every `DATA` and `EVENT` line, that flow, cumulative volume and the float-switch and shutdown behaviour match what the firmware is designed to do, and that the results are the ones expected. The run reported in Sections 3 and 5 is the result of this checking.

10.3 Run history is preserved

Every time the logger is started it creates a new, time-stamped folder (`run_YYYYMMDD_HHMMSS`) inside `case1_case2_runs` or `ball_valve_runs`. Earlier runs are never overwritten, so all runs remain available for side-by-side comparison, and the best run can be selected from the saved samples, plots and reports.

```
case1_case2_runs/
├── run_20261011_230100/
│   ├── serial_log.txt
│   ├── samples.csv
│   ├── events.csv, float_events.csv
│   ├── case1_*.png, case2_*.png, ft01_flow_vs_frequency.png
│   └── Case1_Case2_Experiment_Report.docx
├── run_<next run>/
└── …
```

Folder name shown is an example of the naming pattern.